墨韵书坊：基于 Zynq-7020 的七轴书法机械臂

摘要

本作品面向书法教学与传统文化展示，以正点原子 ATK-DF7020 开发板（Zynq-7020，XC7Z020CLG400-2）为唯一主控，驱动七轴协作机械臂用毛笔、秀丽笔和中性笔书写，并在 7 寸触摸屏上提供名帖表演、屏上手写、照片绘画、手写识字、诗文朗读等 13 种模式。

核心问题是把字形或线稿快速、可靠地变成七个关节的连续轨迹。我们先在板端对整条计算链分阶段计时，发现逆解只占规划时间的 26.9%，由阿姆达尔模型可知单靠加速逆解，规划最多快 1.37 倍；而两个用于数值复核的 FK 校验环节占 44.2%。据此确定两条路线：一是把逆解改为针对 SRS 构型的单精度解析解，设计“几何分解—定点 CORDIC 选解—检查写出”三级 DATAFLOW 核，以“成功前缀 + 首个失败点”协议与 CPU 回退衔接；二是把运行时重复复核移到开发阶段，用独立 FP64 正解做离线验证。

结果：新核在 100 MHz 下 RTL 协同仿真每 64 点 55,043–55,123 周期（约 11.6 万点/秒），21,679 个历史目标连续回放 0 回退、最大位置计算误差 0.39 μm；布线后模块比上一版少 6,139 个 LUT 和 19 个 DSP，整板 LUT 占用由 85.32% 降至 75.28%，建立/保持裕量 +0.136/+0.011 ns，已在板上加载并通过自检。我们也报告了反例：旧数值迭代核接入后，3,434 节点人像的规划由纯 CPU 的 8.20 s 变为 10.84 s，原因是数据搬运占逆解环节的 60%。新平台的整幅规划时间尚待板端实测。开发全程与大模型智能体协作，过程与技能包见附录 A、B，复现步骤见附录 C。

关键词：Zynq-7020；七轴机械臂；解析逆运动学；高层次综合；CORDIC；软硬件协同；书法机器人

第一部分　作品概述

1.1 功能与特性

系统以触摸屏为主入口、手机扫码为辅助入口，共 13 种模式（图 1-1）：

（1）书写类：名帖表演（7 部）、春联、诗词（40 首绝句）、毛笔写字（3,500 个常用字，字号 40–70 mm）、硬笔书法、笔画工坊（88 种笔画模块）；

（2）交互类：屏上手写（手指书写、机械臂复写）、手写识字（3,755 类离线识别）、字迹识别（摄像头比对评分）、照片绘画（拍照→线稿→绘制）；

（3）文化类：书法百科、趣味测验、诗文朗读。

轨迹规划全部在板端完成，云端只在“AI 照片”流程中可选；开始前、结束后自动归位，人工停止为减速软停；PL 求解失败的点由 CPU 接管，书写不中断。



图 1-1　主界面（1024×600，10 月 27 日截图；“手写识字”入口为其后新增）

1.2 应用领域

选题背景：书法是中小学写字课程和传统文化教育的重要内容，但专业教师不足，示范难以重复、放慢和分解。已有书法机器人多在电脑上规划后下发，或依赖控制器自带逆解；七轴冗余臂的逆解通常用雅可比迭代[1,2]，解析解需针对具体构型推导[3]。我们把“字形→轨迹→关节角”全部放进一块 Zynq-7020，让机械臂成为可独立运行、可触屏交互的书写终端。

应用领域：（1）书法与写字教学：慢速示范笔顺与运笔，摄像头比对学生字迹；（2）文化场馆与研学：名帖表演、春联现场书写、诗文朗读伴写；（3）边缘端连续轨

迹作业：七轴规划在嵌入式端完成，可迁移到描线、点胶、标刻等场景。

1.3 主要技术特点

（1）按实测划分软硬件：先剖析整条计算链，再决定什么进 PL；交互、内容、排版和校验在 ARM，连续逆解、显示时序、触摸、网口桥、识别预处理和音频接口在 PL。

（2）SRS 解析逆解硬件化：几何分解与 28 级定点 CORDIC 替代数值迭代，三级 DATAFLOW 并发，FP32 计算并兼容原 FP64 缓冲。

（3）连续分支、可回退：肘部沿上一点方向投影，肩、腕各两支按关节变化平方和最小选取；奇异或超限时只提交成功前缀。

（4）单板集成：显示、触摸、PL 网口、识别核、音频与逆解核共存于一个比特流，Slice 占用 99.71% 下时序收敛。

1.4 主要性能指标

主要指标见表 1-1。“板端实测”指在 ATK-DF7020 上运行所得；“仿真”“离线”为 RTL 协同仿真或电脑端计算，不代表板级端到端时间，也不代表实际落笔精度。

表 1-1 主要性能指标

指标	结果	验证方式
主控器件	XC7Z020CLG400-2	ATK-DF7020
逆解吞吐	约 11.6 万点/s	核级 RTL 仿真
位置计算误差	P99 0.26 μm；最大 0.39 μm	离线 FP64 正解核对
轨迹回退	21,679 个目标，0 回退	离线连续回放
LUT / DSP / BRAM	75.28% / 42.73% / 48.21%	Vivado 布线后
WNS / WHS	+0.136 ns / +0.011 ns	Vivado 布线后
人像规划（CPU / PL）	8.20 s / 10.84 s	v2c 板端均值
线稿提取（前 / 后）	5.88 s / 1.14 s	14 图板端中位数
单笔识别与规划	751 ms	12 组板端中位数
手写识字	3,755 类；单字中位 170 ms	板端实测

测试条件：逆解核为 100 MHz，64 点事务耗时 55,043–55,123 周期；误差为模型内计算误差。人像含 3,434 节点，各模式运行 3 次取平均；线稿输入为 1024×1024。识字常驻内存增加约 49 MB。DRC 为 0 错误。

1.5 主要创新点

（1）在 Zynq-7020 单芯片上完成冗余七轴机械臂从字形到关节轨迹的全流程，无需上位机。

（2）面向 SRS 构型的 FP32 解析逆解核：肘部投影保持连续、分组选支，28 级定点 CORDIC 复用一条通道。

（3）以阶段计时与阿姆达尔模型决定软硬件划分，保留“加速核更快、整体更慢”的反例并据此改向。

（4）将大模型协作沉淀为交接提示词、证据台账与发布校验技能包。

1.6 设计流程

设计流程如图 1-2。先在 CPU 上实现完整算法并测出基线，再按阶段计时找热点，只把数据规则、计算密集的部分做成 HLS 核；每个核依次通过 C 仿真、RTL 协同仿真、Vivado 时序与 DRC、网表检查，封装为 PYNQ 硬件包后按“暂存—校验—激活—回滚”上板。板端实测不达标就回到剖析。

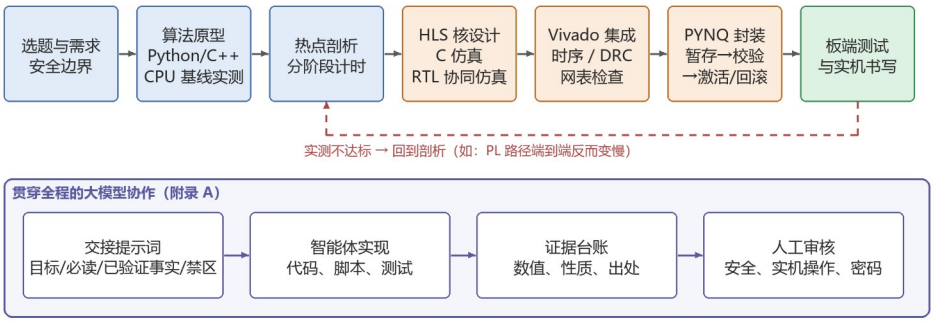


图 1-2　设计流程

第二部分　系统组成及功能说明

2.1 整体介绍

系统组成如图 2-1。ZYNQ-7020 是唯一的计算与控制核心：左侧为输入（触摸屏、手机扫码、USB 摄像头、可选的云端图生图），右侧为输出与执行（LCD、扬声器、机械臂控制器）。PS 端运行 Linux 与 PYNQ[4]，承担交互、内容、规划组织、校验和通信；PL 端承担规则密集的计算和时序严格的接口；二者经 AXI-Lite 控制、经 AXI HP 口共享 DDR[5]。

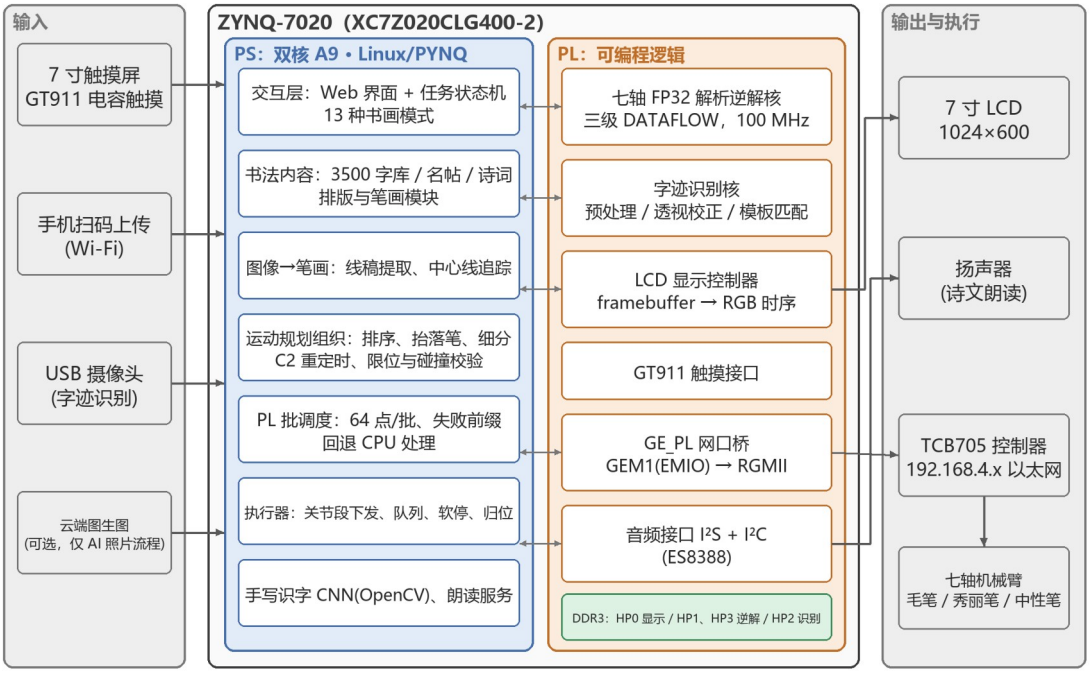


图 2-1　系统整体框图

设计前我们先把需求写成可检验的目标（表 2-1），后文每一项结果都对照这张表报告是否达到。

表 2-1　设计目标与约束

编号	目标与检验
R1	**目标或约束：**全部规划在板端完成，不依赖上位机 **检验方式：**断开电脑后完成书写 **结果所在：**3.1 节
R2	**目标或约束：**逆解与正解在模型内一致：位置 ≤ 0.01 mm，姿态 ≤ 1×10^{-4} rad **检验方式：**独立 FP64 正解逐点核对 **结果所在：**3.3.3 节
R3	**目标或约束：**关节连续：相邻点单关节变化 $\le \Delta q_{max}$，不跳支 **检验方式：**连续回放统计最大步长 **结果所在：**3.3.3 节
R4	**目标或约束：**规划时间可接受：单笔交互 < 1 s，整幅作品数十秒内 **检验方式：**板端阶段计时 **结果所在：**3.3.4、3.3.6 节
R5	**目标或约束：**PL 资源与时序：整板布线后 WNS、WHS ≥ 0，DRC 无错误 **检验方式：**Vivado 报告 **结果所在：**3.3.8 节
R6	**目标或约束：**安全：停机可控、异常即停、不自动清报警 **检验方式：**板端测试 + 模拟控制器 **结果所在：**2.3.2 节

各模块的关系可以按四条路径理解：（1）书写路径：选字 → 排版与坐标变换 → 规划组织目标点 → PL 逆解核批量求解 → CPU 重定时与校验 → 执行器下发 → 机械臂书写；（2）照片绘画路径：手机上传 → （可选）云端生成线稿 → 中心线提取 → 同书写路径；（3）识别路径：手写识字在 CPU 上用卷积网络离线识别；字迹识别由摄像头采图，PL 核做预处理、透视校正和模板匹配；（4）显示与朗读：GUI 由 CPU 渲染，PL 读取 framebuffer 输出 RGB 时序，音频经 PL 的 I²S 接口送至板载 ES8388[6]。

2.2 硬件系统介绍

2.2.1 硬件整体介绍

硬件组成见表 2-2，开发板与显示屏实物见图 3-3、图 3-4。机械臂及控制器为成品设备，我们的硬件工作集中在 PL 逻辑设计、外设连接、笔具安装与标定。

表 2-2 硬件组成

部件	规格与连接
主控板	**型号 / 规格：**正点原子 ATK-DF7020，XC7Z020CLG400-2 **作用：**全部计算、控制与接口 **与主控连接：**—
显示屏	**型号 / 规格：**正点原子 ATK-MD0700R，7 寸 1024×600 RGB **作用：**主界面 **与主控连接：**PL RGB 接口
触摸	**型号 / 规格：**GT911 电容触摸（屏上集成） **作用：**操作、手写输入 **与主控连接：**PL I²C
机械臂	**型号 / 规格：**天链 TCB705 七轴协作臂及控制器 **作用：**书写执行 **与主控连接：**以太网（PL 网口）
音频	**型号 / 规格：**板载 ES8388 编解码器 + 扬声器 **作用：**诗文朗读 **与主控连接：**PL I²S / I²C
摄像头	**型号 / 规格：**USB 摄像头 **作用：**字迹识别采图 **与主控连接：**PS USB
无线	**型号 / 规格：**板端 Wi-Fi（wlan0） **作用：**手机扫码上传 **与主控连接：**PS
书写工具	**型号 / 规格：**毛笔、秀丽笔、中性笔及笔夹 **作用：**书写 **与主控连接：**机械臂末端法兰

2.2.2 机械设计介绍

机械臂为七轴 SRS（球—转—球）构型：关节 1–3 轴线交于肩点，关节 5–7 轴线交于腕点，关节 4 为肘。运动学参数直接从控制器变量 r_dh_1_ 读出（固件 1.9.71，10 月 18 日），见表 2-3；全部关节零偏为 0，连杆长度 a_i 为 0，这正是解析逆解得以成立的结构条件。

表 2-3　TCB705 标准 DH 参数与关节限位（取自控制器）

关节	d_i / mm	α_i / °	a_i / mm	限位 / °
1	263.0（d_{bs}）	90	0	±178.9
2	0	−90	0	±134.9
3	320.5（d_{se}）	90	0	±178.9
4	0	−90	0	±143.0
5	279.5（d_{ew}）	90	0	±178.9
6	0	−90	0	±133.0
7	161.5（d_{wf}）	0	0	±178.9

在此基础上我们完成了四项机械相关工作：（1）笔具安装：毛笔、秀丽笔和中性笔经笔夹竖直装在法兰上（图 3-7），工具长度并入 d_t；（2）高度标定：竖直笔落笔时法兰高度 $Z = 219.3$ mm，归位点在纸面中心上方 $Z = 319.3$ mm；（3）运动学标定：拟合各关节符号、零位、基座与工具中心点，留出样本检验不超过 0.5 mm / 0.2°，并拒绝镜像解；（4）纸面标定：标定纸面原点与两轴，书写区 148.52 mm × 150.215 mm。

2.2.3 电路各模块介绍

PL 部分的模块连接如图 2-2，各模块资源见表 2-4（10 月 27 日完整平台布线后层次化报告）。

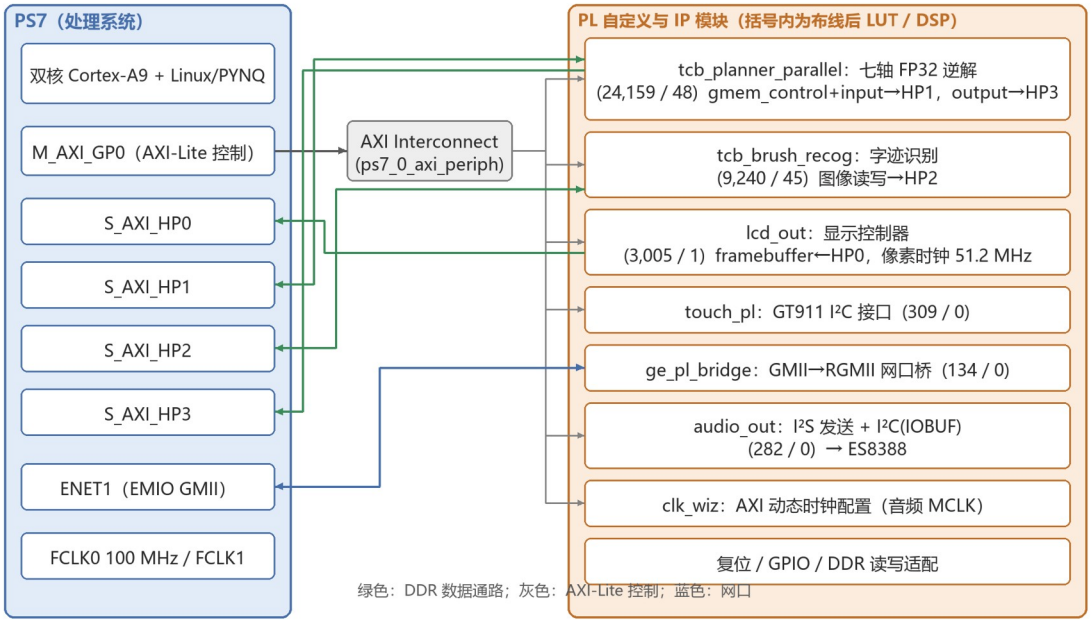


图 2-2　PL 模块连接与数据通路

表 2-4　PL 主要模块资源（布线后）

模块	LUT	FF	BRAM36	DSP
逆解核	24,159	35,279	20	48
识别核	9,240	10,473	36.5	45
显示控制	3,005	5,488	3	1
音频时钟	1,073	1,488	0	0
音频接口	282	396	8	0
触摸接口	309	324	0	0
网口桥	134	235	0	0
平台合计	40,048	56,343	67.5	94

模块对应：逆解核 tcb_planner_parallel；识别核 tcb_brush_recog；显示 lcd_out；音频时钟 clk_wiz；音频 audio_out；触摸 touch_pl；网口桥 ge_pl_bridge。平台合计包含 AXI 互联等资源。

（1）七轴逆解核：控制与目标读取经 HP1，结果写回经 HP3，时钟 100 MHz，结构见 2.5 节。（2）字迹识别核：USB 摄像头图像的预处理、透视校正和 3,500 字模板匹配，经 HP2 读写 DDR。（3）LCD 显示控制器：从 HP0 读取 framebuffer，生成 1024×600 液晶时序，像素时钟 51.2 MHz。（4）GT911 触摸接口：PL 实现 I²C 读取触点[7]，CPU 转换为输入事件。（5）GE_PL 网口桥：PS 的 GEM1 经 EMIO 引出 GMII，由 PL 转为 RGMII 直连机械臂控制器；IP/TCP 与控制器协议仍在 CPU。（6）音频接口：I²S 发送 + I²C 配置 ES8388。首版网表中 SDA 的输入与三态缓冲被综合掉，总线常低并产生虚假应答；改为显式 IOBUF 后布线网表中 SDA 为 IBUF + OBUFT，并新增“网表缺缓冲即拒绝封装”的检查。

2.3 软件系统介绍

2.3.1 软件整体介绍

软件分为六层（图 2-3）。板端以 systemd 服务运行：tcb705-ui（后端与界面）、显示服务、触摸服务，以及开机加载 FPGA 平台的 platform 服务。界面是本地网页，运行在板载 WebKitGTK 2.28 中，脚本和样式避开该内核不支持的新语法，由测试自动检查。

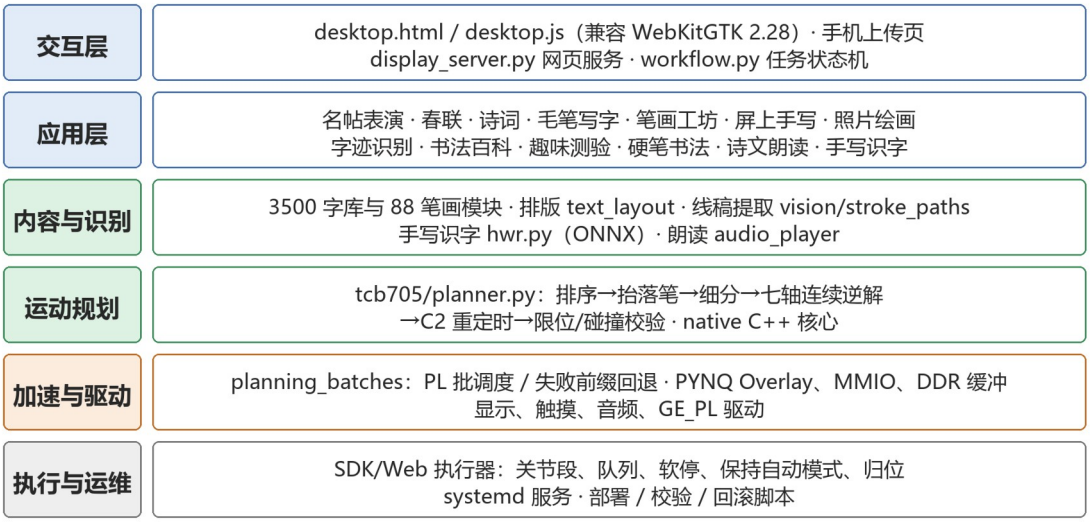


图 2-3　软件分层

2.3.2 软件各模块介绍

（1）运动规划模块（tcb705/planner.py，图 2-4）。输入为字库笔画、线稿中心线或触屏笔迹；先排版并变换到纸面坐标，再排序笔画、加入抬笔/落笔和空移节点，按 0.5 mm（毛笔落纸段不超过 0.4 mm）细分为目标点，交给七轴连续逆解（2.5 节）；之后做 C2 五次曲线重定时、关节限位与连续性校验，最终合并为控制器可执行的关节段。

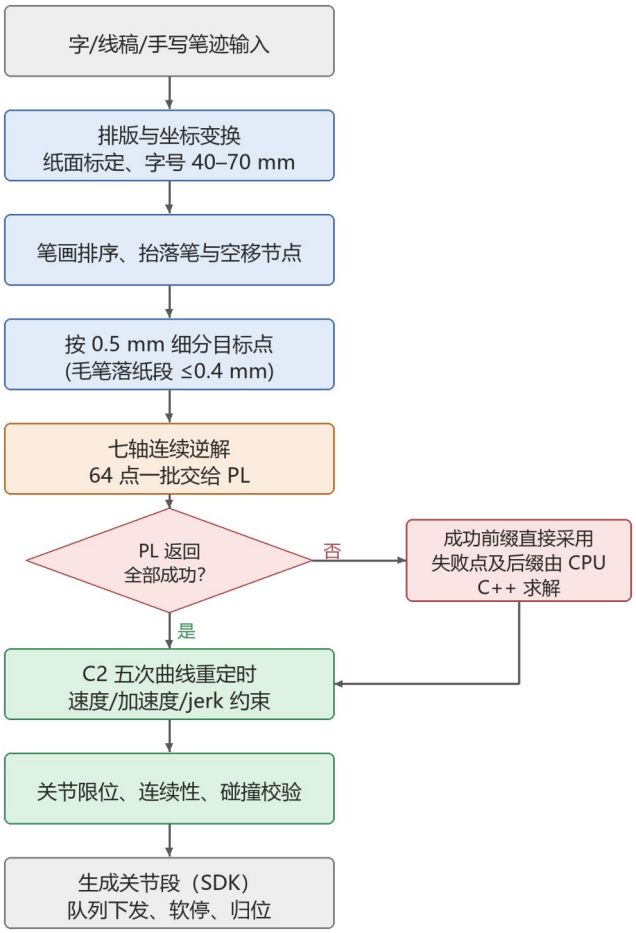


图 2-4　运动规划流程

（2）书法内容与排版模块。字库含 3,500 个常用字、33,959 笔（约 26.4 MiB，常驻内存）；笔画工坊提供 88 种笔画模块。排版支持长文本按字号自动换行、分页，每页结束后归位并提示换纸。名帖 7 部共 21 列的规划结果已在板端预计算缓存。

（3）执行与运动控制模块（tcb705/web_executor.py）。板上程序与控制器之间有两条通道（图 2-5）：华成 SDK（V2.4.1）只负责下发运动；控制器 Web API（HTTP + JSON）负责登录、读取状态与位姿、切换模式和读取报警。SDK 库 libxoip.so 是 ARM32 动态库、没有 Python 接口，因此用一个 C++ 桥接程序 tcb_sdk_server：Python 经标准输入写一行文本命令，桥接程序调用 SDK 函数并返回一行结果，运动完成事件由桥接程序在回调中主动输出。采用关节模式：关节角由板上规划与 PL 逆解给出，控制器只负责插补和伺服驱动。

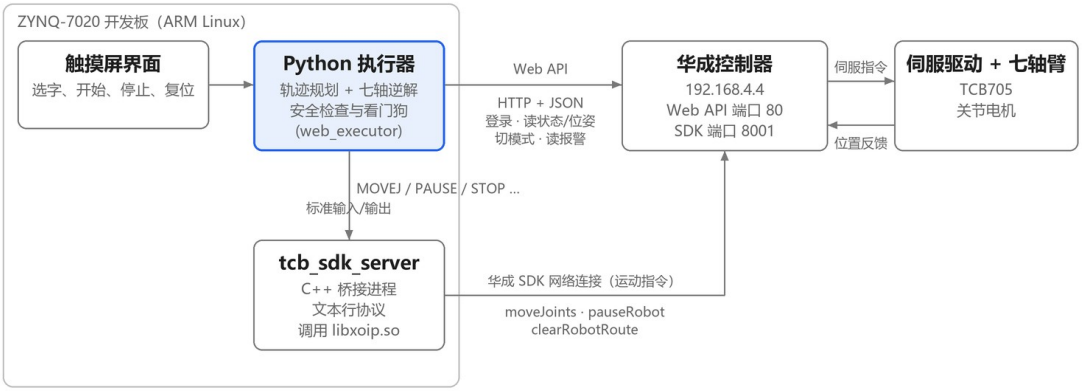


图 2-5　机械臂控制链路（七轴逆解由 PL 核完成，Python 执行器负责调度与下发）

一次书写任务的指令流程是：①每个路径点生成一个关节运动块（hmcRobotFree，7 个关节角，掩码 127），相邻共线点合并；②启动桥接程序并 CONNECT 控制器，Web API 登录，确认轴数为 7、无报警无急停，当前位姿与规划起点相差不超过 3 mm（各关节不超过 0.5°）；③若不在自动模式，在笔尖离纸至少 60 mm 时切换并上伺服，等待静止且上电位移不超过 10 mm；④ SPEED 写入界面速度，归位按“竖直抬高 → 平移到原点上方 → 关节运动到原点”执行，避免笔在纸面附近横向拖动；⑤按算法 1 逐段下发 MOVEJ；⑥每 0.1 s 读一次末端位姿，偏离规划折线超过 3 mm、低于规划最低点余量（默认 0.5 mm，硬笔不少于 1.0 mm）、连续 10 s 无进展，或 SDK 报告失败、拒绝、桥接进程退出，立即停止；⑦写完回原点，停止程序运行键，控制器保持自动模式等待下一任务。

算法 1　在途指令队列与完成确认

输入：运动块序列 {m_1, …, m_M}，队列深度 Q（毛笔 1，屏幕手写 4）
1: n_fly ← 0
2: for j = 1 to M do
　3: 读取已到达的 EVT DONE 事件，n_fly 相应减少；若任一结果 ≠ 1，STOP 1 并退出
　4: if smooth(m_j) = 0 then 等待 n_fly = 0　　// 精确到位指令会打断前一路径
　5: while n_fly ≥ Q do 等待 EVT DONE
　6: seq ← MOVEJ(m_j)
　7: if seq = 0 then 视为同步完成
　8: else if seq ≥ 10 then n_fly ← n_fly + 1
　9: else STOP 1 并退出（不重发被拒绝的指令）
　10: if smooth(m_j) = 0 then 等待 n_fly = 0
　11: 每发送若干条后让出控制权，供监视循环检查取消与位置
12: end for；等待 n_fly = 0

执行器还处理了三类起停抖动。控制器对每段运动采用梯形速度规划，加速段加速度如式（2-1）所示，SDK 只提供 0.1–1.0 s 的加、减速时间，不提供 jerk 设置。因此：

起步与归位的空中运动把 t_a 设为 0.6 s，书写段保持控制器默认；屏幕“停止”先发 PAUSE 1 按减速曲线停稳（最多等 2 s），再 STOP 1 清除剩余路径、PAUSE 0 解除暂停（图 2-6），故障停止不经减速直接清除，并经 Web API 再清一次；控制器手动/自动切换会下沉 7–15 mm，因此任务之间保持自动模式。

$$a=\frac{v}{t_a} \tag{2-1}$$

式中 v 为目标速度，t_a 为加速时间。手写模式另采用平滑等级 3、20° 转角阈值，只平滑书写段：测试中平滑空中段会使轨迹偏离 23.75 mm，因此空中段保持精确到位。相关板端测试 91 项全部通过。

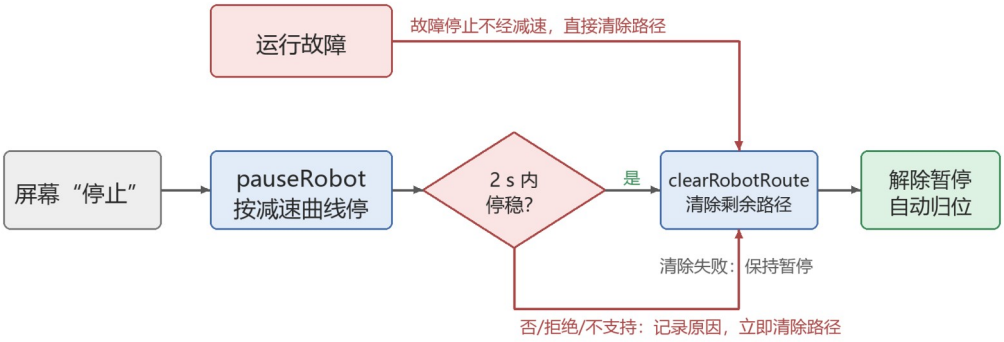


图 2-6　减速软停流程

（4）手写识字模块（user_gui/hwr.py）。采用开源 MobileNetV2[8] 手写汉字模型（CASIA-HWDB1.1[9] 训练，3,755 类），导出为 ONNX[10]（opset 9），由板上已有的 OpenCV 4.2[11] 推理。触屏笔迹按字自身大小渲染，缩放系数见式（2-2），线宽取字长边的 4.5%；归一化与输出概率见式（2-3）、式（2-4）。

$$s=\frac{0.8\times 256}{max\left(w,h\right)} \tag{2-2}$$

$$x=\frac{255-I}{255} \tag{2-3}$$

$$p_i=\frac{\exp\left(z_i-\max\limits_{j} z_j\right)}{\sum\limits_{k=1}^{3755}\exp\left(z_k-\max\limits_{j} z_j\right)} \tag{2-4}$$

式中 w、h 为笔迹外接框宽和高（坐标归一化到 0–1），I 为 64×64 归一化灰度图，z_i 为第 i 类的输出值。该模型计算量小，PL 已近满载，因此留在 CPU。

（5）诗文朗读模块（audio_player）。提供 40 首绝句与 7 部名帖全文录音、释文和分句时间轴。I²C 地址探测确认编解码器为 ES8388（0x10）；48 kHz 音频按 12.5 MHz MCLK/256 对应的 48,828.125 Hz 重采样，保持语速与音调。62 项板端测试通过；

扬声器实际出声尚待验收。

（6）照片绘画模块。手机与板子接入同一 Wi-Fi 后扫码上传，云端只用于生成线稿，上传现成线稿可跳过云端。板端用细化[12]和图追踪提取中心线，再以 Savitzky–Golay 平滑（窗口 7、二阶、容差 0.2 mm、保留 70° 以上拐角）。当前平台为容纳识别核与 FP32 核移除了细化 IP，软件检测到后自动改用 CPU。

2.4 软硬件划分依据与接口设计

2.4.1 划分依据

划分不凭直觉，而是先对整条计算链在板上分阶段计时（3.3.4 节），再用阿姆达尔模型[13]估计每个环节加速的收益上限。设某环节占总时间的比例为 f、该环节被加速 s_k 倍，则总加速比满足式（2-5）。

$$S=\frac{1}{(1-f)+f/s_k}\le\frac{1}{1-f} \tag{2-5}$$

板端实测中，纯 CPU 规划 8.18 s 里逆解环节占 2.20 s（$f=26.9\%$），即使逆解无限快，规划也只能快到 1.37 倍；而“曲线 FK 校验”和“4 ms 帧校验”两个复核环节合计 3.61 s（44.2%）。图 2-7 把这一关系画了出来：只加速逆解时曲线很快逼近 5.98 s 的下限，同时去除两项运行时复核后下限降到 2.36 s。这决定了我们的两条改进路线：逆解改为解析解并硬件化，运行时复核移到开发阶段做离线验证（2.5.1 节假设 H4）。具体划分见表 2-5。

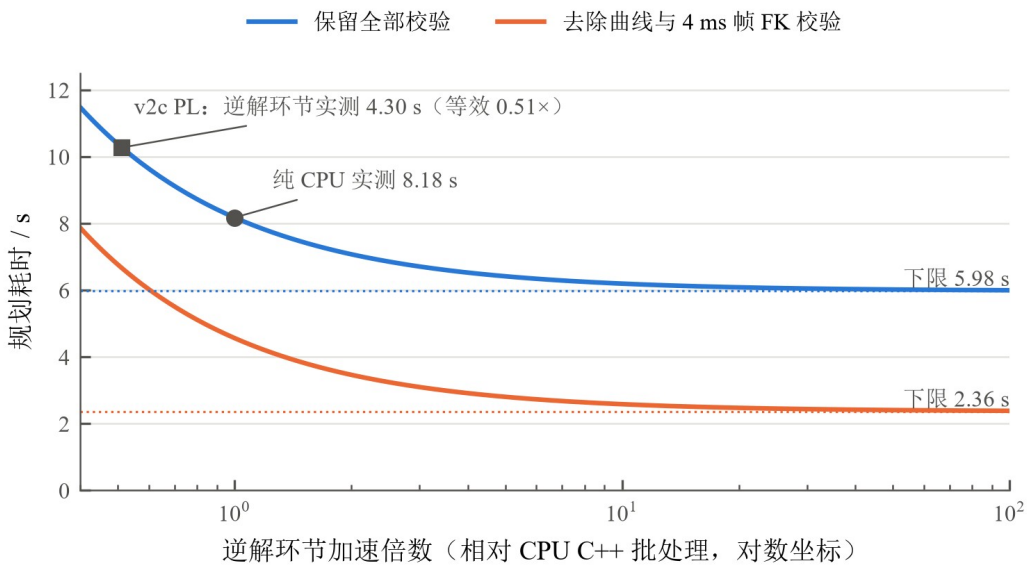


图 2-7　规划耗时随逆解加速倍数的变化（阿姆达尔模型，参数取自板端实测）

表 2-5 软硬件划分及依据

任务	分工依据
连续七轴逆解	**位置：**PL **依据：**目标点数千至数万，运算规则一致，可 64 点成批流水；解析形式只有乘加、开方与反正切
重定时、限位与碰撞校验	**位置：**CPU（C++） **依据：**依赖全局轨迹与状态，分支多；C++ 批处理后已较快
线稿细化	**位置：**CPU（C++/OpenCV） **依据：**PL 细化中位 0.117 s（C++ 0.318 s），但为识别核与 FP32 核让出 LUT 而移除
显示、触摸、网口桥、I²S	**位置：**PL **依据：**像素/比特级时序严格
字迹识别预处理与匹配	**位置：**PL **依据：**像素级规则运算，数据量大
手写识字 CNN	**位置：**CPU **依据：**64×64 小模型 170 ms 已满足交互；PL 资源已近满
界面、云端、控制器通信	**位置：**CPU **依据：**协议与业务逻辑，变化频繁

2.4.2 接口设计

逆解核的软硬件接口见表 2-6。PS 通过 AXI-Lite 写入各缓冲区物理地址和参数后启动核；核经三个 AXI 主接口直接读写 DDR。取消字在核运行中实时读取，用户按“停止”时可中止长批。完成头包含状态、已完成点数和首个失败点序号；能力字 capabilities = 3 表示 FP32 且“误差未评估”，主机把误差字段（quiet-NaN）转为 null，不把未测量的误差显示为 0。

表 2-6 七轴逆解核接口

编号	接口定义
01	**接口：**control **类型 / 偏移：**AXI-Lite，0x10–0x6C **内容：**config/input/output/result/cancel 缓冲地址，mode、count、output_capacity、job_id、generation
02	**接口：**gmem_control **类型 / 偏移：**M_AXI → HP1 **内容：**模型参数、初始姿态、取消字（运行中读取）
03	**接口：**gmem_input **类型 / 偏移：**M_AXI → HP1，突发 64 **内容：**目标位姿（沿用 FP64 打包，边界处转 FP32）
04	**接口：**gmem_output **类型 / 偏移：**M_AXI → HP3，突发 64 **内容：**七个关节角、误差字段、完成头（状态/完成数/首个失败点）
05	**接口：**协议约束 **类型 / 偏移：**— **内容：**只支持 IK_CHAIN；FK_ROWS 明确拒绝；返回成功前缀，CPU 处理后缀

与控制器的接口：PL 网口以静态地址直连控制器（192.168.4.4，Web API 端口 80，SDK 端口 8001，机器人编号 1）。SDK 通道采用表 2-7 所列文本行协议，回复以 OK 或 ERR 开头。MOVEJ 返回的序号用于对应完成事件：不小于 10 表示已异步排队、等待 EVT DONE；0 表示已同步完成；其余值表示控制器拒绝，执行器立即停止且不重发。实测该控制器经 SDK 读取关节反馈没有数据，因此位置与状态统一从 Web API 读取（表 2-8）。

表 2-7　SDK 桥接命令（tcb_sdk_server）

编号	格式与作用
01	**命令：**CONNECT **格式：**CONNECT 本机 IP 控制器 IP 端口 机器人号 **SDK 函数：**init_data **作用：**建立连接，返回库与控制器版本
02	**命令：**SPEED / GETSPEED **格式：**SPEED 0..100 **SDK 函数：**setGlobalSpeed / getGlobalSpeed **作用：**设置 / 读取全局速度百分比
03	**命令：**MOVEJ **格式：**MOVEJ 平滑 加速 减速 比例 q1..q7 **SDK 函数：**moveJoints **作用：**关节空间运动到 7 个关节角（度），返回序号
04	**命令：**MOVEL **格式：**MOVEL 平滑 加速 减速 比例 x y z u v w **SDK 函数：**moveLine **作用：**末端直线运动（mm、度），用于归位
05	**命令：**STATE **格式：**STATE **SDK 函数：**getRobotMoveState 等 **作用：**运动状态、是否暂停、链路是否正常
06	**命令：**PAUSE **格式：**PAUSE 1 / PAUSE 0 **SDK 函数：**pauseRobot **作用：**按减速曲线暂停 / 解除暂停
07	**命令：**STOP **格式：**STOP 1 **SDK 函数：**clearRobotRoute **作用：**清除控制器中剩余全部路径
08	**命令：**ALARM **格式：**ALARM **SDK 函数：**isAlarming **作用：**只读报警，不清除

完成事件：桥接程序主动输出 EVT DONE 序号 结果 机器人号，由 SDK 运动结束回调触发；结果 1 表示正常完成。

运动指令参数：平滑 0 为精确到位（会打断前一路径），1–9 为与下一段平滑过渡；加、减速时间 0 表示控制器默认，否则取 0.1–1.0 s；比例固定为 0；关节角绝对值不超过 360°，非有限数或越界值由桥接程序直接拒绝。

表 2-8 Web API 接口与主要控制器变量

编号	定义与用途
01	**接口 / 变量：**POST /api/user/login **用途：**登录取得令牌，之后每个请求在 Authorization 头携带
02	**接口 / 变量：**GET /api/var/byname **用途：**按名称读取变量：状态、末端位姿、关节角等
03	**接口 / 变量：**POST /api/power **用途：**{"mode":"auto"} 自动并上伺服；{"mode":"manual"} 手动
04	**接口 / 变量：**GET /api/log/alarm **用途：**读取报警记录（history = 0 为当前报警）
05	**接口 / 变量：**POST /api/robot/clear **用途：**清除机器人路径，作为 SDK 停止的后备
06	**接口 / 变量：**/api/mold/download、/api/mold/shift **用途：**下载并切换控制器程序（备用执行方式）
07	**接口 / 变量：**_phc_ctrlkey / _phc_key **用途：**控制模式（100 = 自动）/ 程序运行键（100 运行，200 停止）
08	**接口 / 变量：**g_servo_statero_ / _phc_alarm_ / g_state_ **用途：**伺服已上电 = 1 / 无报警 = 0 / 未急停 = 1
09	**接口 / 变量：**r_axis_num_1_ **用途：**机器人 1 的轴数，必须为 7
10	**接口 / 变量：**r_world_fb_pos_1_0…5_ / r_joint_fb_pos_1_0…6_ **用途：**末端位姿反馈（mm、度）/ 7 个关节角反馈（度）

SDK 通道不可用时，可把全部运动块打包为控制器程序，经 /api/mold/download 下载、/api/mold/shift 切换后置运行键为 100 执行。这种方式速度只能整体设定、停止只能清除路径，仅作后备。

2.5 关键算法设计与理论分析

2.5.1 模型假设与符号说明

逆解与轨迹模型基于表 2-9 所列假设。每条假设都给出依据，并说明它在后文哪里被使用或检验。

表 2-9　模型假设

编号	假设与依据
H1	**假设：**机械臂为刚体，运动学由表 2-3 的标准 DH 参数完整描述，忽略连杆柔性与关节间隙 **依据：**参数直接读自控制器；控制器 DH 模型与关节反馈相差 ≤ 0.02 mm（10 月 18 日） **使用 / 检验：**式（2-6）至式（2-15）
H2	**假设：**笔轴竖直，工具变换只含沿法兰 z 轴的偏移 d_t **依据：**笔夹竖直安装，规划姿态固定为竖直 **使用 / 检验：**式（2-8）
H3	**假设：**纸面为平面，由原点与两个单位轴确定 **依据：**纸面标定；书写区 148.52 mm × 150.215 mm **使用 / 检验：**2.2.2 节
H4	**假设：**开发阶段已用独立 FP64 正解验证的核，运行时不再逐点复核 **依据：**21,679 点连续回放全部满足 FP32 预算 **使用 / 检验：**3.3.3 节；运行时仍保留限位、步长与有限值检查
H5	**假设：**控制器关节插补误差不计入逆解计算误差 **依据：**两者来源不同，前者需实物测量 **使用 / 检验：**3.3.3 节只报告模型内误差

主要符号见表 2-10，其余符号在首次出现处说明。

表 2-10　符号说明

符号	含义	单位
$q=(\theta_1,\dots,\theta_7)$	关节角向量	rad
p_t，R_t	笔尖目标位置与姿态	m，—
d_{bs}，d_{se}，d_{ew}，d_t	基座—肩、肩—肘、肘—腕、腕—笔尖距离	m
p_{sw}，D，u	肩到腕的向量、其长度与单位方向	m，m，—
a，h	肘部圆心到肩的距离、肘部圆半径	m
e_k	第 k 个目标点的肘部位置（相对肩点）	m
Δq_{max}	相邻点允许的单关节最大变化	rad
B，N，C	批长度（64）、批内有效点数、事务时钟周期数	—
f_{clk}	逆解核时钟频率（100 MHz）	Hz
f，s_k，S	被加速环节占比、该环节加速倍数、总加速比	—

2.5.2 运动学模型

采用标准 DH 约定[14]，记 T_b^a 为坐标系 b 到坐标系 a 的齐次变换，相邻连杆的齐次变换见式（2-6），笔尖位姿由式（2-7）给出。由表 2-3 可知 $a_i=0$、$\alpha_i=\pm 90^\circ$，关节 1–3 与 5–7 分别构成球腕型的肩和腕。

$$T_i^{i-1}=Rot_z(\theta_i)Trans_z(d_i)Trans_x(a_i)Rot_x(\alpha_i) \tag{2-6}$$

$$T_t^0=\left(\prod_{i=1}^{7}T_i^{i-1}\right)T_t^7=\begin{bmatrix}R_t & p_t\\ 0^T & 1\end{bmatrix} \tag{2-7}$$

2.5.3 SRS 构型的解析逆解

七轴臂对一个笔尖位姿有一维冗余（肘部可绕肩—腕连线转动）。数值方法用阻尼最小二乘迭代[1,2]，每点需多次求解 7×7 方程，难以在 PL 上流水；解析方法则把冗余显式参数化[3]。我们的做法是：冗余参数不额外引入，而是取“上一点的肘部方向”，使肘部在连续目标之间转动最小。推导分五步。

第一步，求腕点。由 H2，腕点等于笔尖沿笔轴回退 d_t，肩到腕的向量见式（2-8），可达条件见式（2-9）。

$$\begin{aligned} p_{sw}&=p_t-d_tR_te_z-d_{bs}e_z\\ D&=\|p_{sw}\|,\ u=p_{sw}/D \end{aligned} \tag{2-8}$$

$$|d_{se}-d_{ew}|<D<d_{se}+d_{ew} \tag{2-9}$$

第二步，求肘角。肩、肘、腕构成三角形，肘点位于以 $a\,u$ 为圆心、半径 h 的圆上，见式（2-10），肘角由余弦定理给出，见式（2-11），其中 σ 取上一点 $\sin\theta_4$ 的符号，保证肘部不翻转。

$$a=\frac{d_{se}^2-d_{ew}^2+D^2}{2\,D},\ h=\sqrt{d_{se}^2-a^2} \tag{2-10}$$

$$\begin{aligned}\cos\theta_4&=\frac{D^2-d_{se}^2-d_{ew}^2}{2\,d_{se}d_{ew}}\\ \sin\theta_4&=\sigma\frac{D\,h}{d_{se}d_{ew}}\end{aligned} \tag{2-11}$$

第三步，确定肘部位置。把上一点肘部 e_{k-1} 投影到与 u 垂直的平面，再放到圆上，见式（2-12）。这一步就是冗余的解：它在所有可行肘部中选取与上一点方向最接近的一个。

$$\begin{aligned}v&=e_{k-1}-\left(e_{k-1}^T u\right)u\\ e_k&=a\,u+h\frac{v}{\|v\|}\end{aligned} \tag{2-12}$$

第四步，求肩部三角。肩部坐标系三轴由肘部方向和前臂方向构造，见式（2-13），记 $R_s=[x_s\,y_s\,z_s]=(r_{ij})$，则关节 1–3 由式（2-14）给出。

$$\begin{aligned}z_s&=\frac{e_k}{d_{se}},\ y_s=z_s\times x_s\\ x_s&=\frac{1}{\sin\theta_4}\left(\cos\theta_4\,z_s-\frac{p_{sw}-e_k}{d_{ew}}\right)\end{aligned} \tag{2-13}$$

$$\begin{aligned}\theta_1&=\operatorname{atan2}\left(r_{12},r_{02}\right)\\ \theta_2&=-\operatorname{atan2}\left(\sqrt{r_{02}^2+r_{12}^2},r_{22}\right)\\ \theta_3&=\operatorname{atan2}\left(r_{21},-r_{20}\right)\end{aligned} \tag{2-14}$$

第五步，求腕部三角。前臂坐标系为 R_s 绕 θ_4 旋转后的 R_4，腕部相对旋转 $W=R_4^T R_t=(w_{ij})$，关节 5–7 的形式与肩部相同，见式（2-15）。实现中只计算 W 的五个元素，避免两次完整的 3×3 矩阵乘。

$$\begin{aligned}\theta_5&=\operatorname{atan2}\left(w_{12},w_{02}\right)\\ \theta_6&=-\operatorname{atan2}\left(\sqrt{w_{02}^2+w_{12}^2},w_{22}\right)\\ \theta_7&=\operatorname{atan2}\left(w_{21},-w_{20}\right)\end{aligned} \tag{2-15}$$

分支选择与奇异保护。肩部和腕部各有两支解，第二支为 $(\theta\pi,-\theta,\theta\pi)$。两组的代价互相独立，因此分别在两支中选取满足限位与步长约束、且关节变化平方和最小的一支，见式（2-16），只需 13 次关节检查，而联合枚举四种组合需要 28 次，结果相同。当肘、肩或腕接近奇异时解析式数值病态，因此加入式（2-17）的条件门槛，不满足时

返回失败点，由 CPU 的数值解处理。门槛 0.05 是用 FP64 参考解离线试验选定的求解域限制，不是放宽验收误差。

$$q_g^{*}=\arg\min_{b\in\{0,1\}}\sum_{j\in g}\left(q_j^{(b)}-q_{j,k-1}\right)^2$$
$$\text{s.t.}\ q_j^{min}\le q_j^{(b)}\le q_j^{max},\left|q_j^{(b)}-q_{j,k-1}\right|\le\Delta q_{max} \tag{2-16}$$

$$\left|\sin\theta_4\right|\ge 0.05,\left\|v\right\|\ge 0.05\,d_{se}$$
$$\sqrt{r_{02}^2+r_{12}^2}\ge 0.05,\sqrt{w_{02}^2+w_{12}^2}\ge 0.05 \tag{2-17}$$

式（2-16）中 g 为肩组 $\{1,2,3\}$ 或腕组 $\{5,6,7\}$，b 为分支编号。整个求解只含乘加、一次除法、三次开方和七次反正切，正适合硬件流水。

2.5.4 定点 CORDIC 反正切

七个关节角都由 atan2 得到。若用浮点反正切 IP，每个实例都要占用较多 DSP 与 LUT；我们改用向量模式 CORDIC[15,16]，迭代见式（2-18），只需移位与加减。输入 FP32 先按指数对齐成 Q3.28 定点数（32 位有符号），迭代 28 次，角度表 28 项存于 LUTRAM。

$$x_{i+1}=x_i-\delta_i 2^{-i}y_i,\ y_{i+1}=y_i+\delta_i 2^{-i}x_i$$
$$z_{i+1}=z_i-\delta_i\arctan 2^{-i},\ \delta_i=-\operatorname{sign}(y_i) \tag{2-18}$$

迭代结束时 y 趋于 0，z 收敛到 $\operatorname{atan2}(y_0,x_0)$，迭代 $n=28$ 次时截断误差的量级见式（2-19），再叠加 Q3.28 的量化步长 $2^{-28}\approx 3.7\times10^{-9}$ rad，均远小于 FP32 预算 1×10^{-4} rad。对 1,050,000 组输入的测试中，1,041,041 组与原三通道实现逐位一致，其余 8,959 组为无效输入，按协议拒绝。

$$\left|z_n-\operatorname{atan2}(y_0,x_0)\right|\lesssim\arctan 2^{-(n-1)}\approx 2^{-27}\approx 7.45\times10^{-9}\,rad \tag{2-19}$$

2.5.5 轨迹重定时

逆解得到的关节节点之间用五次多项式连接，见式（2-20），两端位置、速度、加速度共六个边界条件唯一确定六个系数，保证轨迹 C2 连续[17]。每段时长 T 取满足各关节速度、加速度与 jerk 上限的最小值，见式（2-21）。8 笔人像测试中，七个关节的峰值 jerk 均不超过 3.95 rad/s³。

$$q_j(t)=\sum_{m=0}^{5}c_{j,m}t^m,\ t\in[0,T] \tag{2-20}$$

$$T^{*}=\min T\ \text{s.t.}\ \max_t\left|\dot q_j\right|\le\dot q_j^{max},\ \max_t\left|\ddot q_j\right|\le\ddot q_j^{max}$$
$$\max_t\left|\dddot q_j\right|\le\dddot q_j^{max} \tag{2-21}$$

2.5.6 HLS 硬件架构与吞吐分析

逆解核用 Vitis HLS[18]实现，结构如图 2-8：三个任务以深度 2 的 FIFO 相连，在 DATAFLOW 下并发。资源与吞吐之间的取舍见表 2-11。

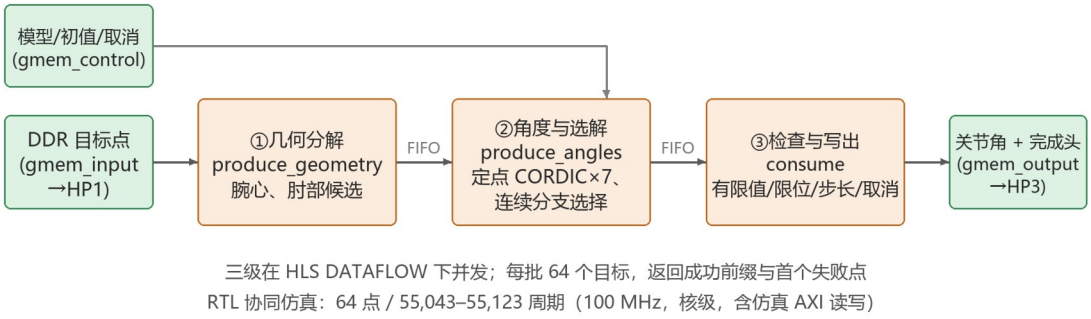


图 2-8　FP32 七轴逆解核数据流

表 2-11　主要 HLS 优化指令与取舍

编号	实现与取舍
01	**指令 / 做法：**DATAFLOW + STREAM depth=2 **位置：**chain() **作用与代价：**几何、选解、写出三级并发；FIFO 浅，BRAM 占用小
02	**指令 / 做法：**ALLOCATION fmul/fadd/fsub limit=2（选支处 limit=1） **位置：**顶层与 select_branch **作用与代价：**限制浮点单元数量换面积；单点延迟变长但吞吐由流水保证
03	**指令 / 做法：**一条 CORDIC 通道复用 7 次 **位置：**produce_angles **作用与代价：**原三通道版本面积更大；复用后角度阶段变长，换取整板空间
04	**指令 / 做法：**PIPELINE II=1 + ARRAY_PARTITION complete **位置：**向量与 3×3 矩阵循环 **作用与代价：**小数组拆为寄存器，逐元素流水
05	**指令 / 做法：**m_axi 突发 64、outstanding 4 **位置：**gmem_input / gmem_output **作用与代价：**成批读写 DDR，减少总线事务
06	**指令 / 做法：**FP32 计算、FP64 打包接口 **位置：**边界转换 **作用与代价：**沿用旧缓冲布局，主机端代码改动最小；传输量未减半

核级吞吐由式（2-22）计算。代入协同仿真的最慢事务（$N=64$，$C=55\,123$）得约 1.16×10^{5} 点/秒，即每点约 8.6 μs。该值包含仿真 AXI 模型下的初始化与读写，但不含

PS 端调度、缓存维护与 DDR 争用，因此不能直接当作整幅规划时间。

$$R=\frac{N f_{clk}}{C}=\frac{64\times 100\,MHz}{55\,123}\approx 1.16\times 10^{5}\,s^{-1} \tag{2-22}$$

2.5.7 批处理协议与 CPU 回退

PL 只交付“成功前缀 + 首个失败点”，CPU 从失败点接着算，下一批以最后一个已接受的关节角为种子，流程见算法 2。这样硬件不必处理奇异、超限等少见情况，也不会把未验证的结果交给执行器。

算法 2　PL 批处理逆解与前缀回退

输入：目标位姿序列 {T_1, …, T_n}，初始关节角 q_0，批长 B = 64

输出：关节角序列 {q_1, …, q_n}

1: k ← 1；q_seed ← q_0

2: while k ≤ n do

3: b ← min(B, n − k + 1)；把 T_k … T_{k+b−1} 写入输入缓冲

4: 写 count = b、job_id、generation，启动核；等待完成头 (status, done, first)

5: 接受 q_k … q_{k+done−1}　　// 成功前缀

6: if done < b then 用 CPU 数值解从 first 起求出本批剩余目标

7: 运行时检查：有限值、关节限位、单步变化 ≤ Δq_max

8: q_seed ← 本批最后一个已接受的关节角；k ← k + b

9: if 用户取消 then 写取消字，丢弃未提交结果并退出

10: end while

第三部分　完成情况及性能参数

3.1 系统整体介绍



图 3-1　系统书写工位全景：七轴机械臂、笔具夹持机构与纸面作业区

系统由七轴机械臂、ATK-DF7020 主控板、7 寸触摸屏和板端交互软件组成。用户在触摸屏选择内容或通过手机上传，开发板完成字形处理、轨迹规划和逆解，再向机械臂控制器下发关节运动段。图 3-1 展示实际书写工位；主控板与显示屏的实物分别见图 3-3、图 3-4。

3.1.1 机械臂本体

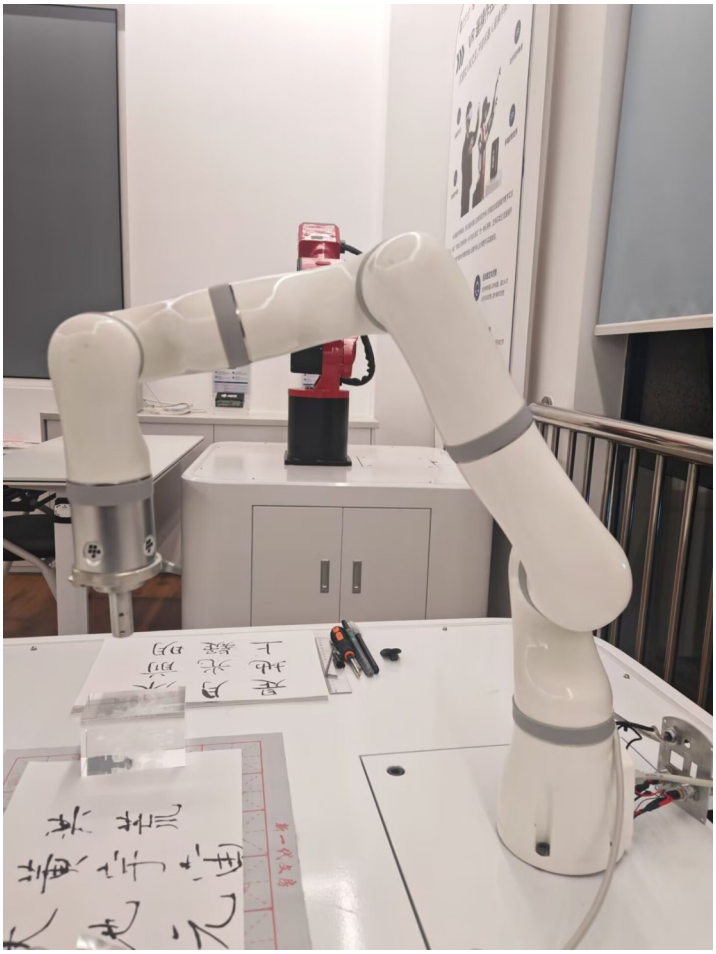


图 3-2　七轴机械臂本体及毛笔书写现场

机械臂采用七轴 SRS 构型，通过末端夹持机构安装书写工具。毛笔用于表现笔画粗细和运笔变化，秀丽笔与中性笔用于硬笔字及线条绘制。更换笔具后重新标定笔尖与落笔高度；任务开始前和完成后执行归位。纸面作业区采用书写纸与镇纸固定，现场已完成的作品见 3.2 节。

3.1.2 主控板及电路

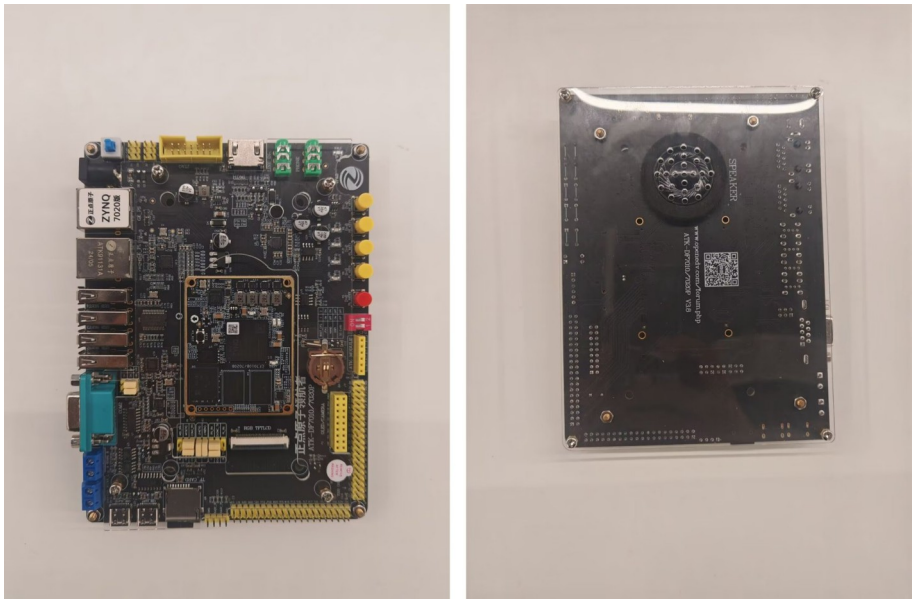


图 3-3 ATK-DF7020 主控板正反面

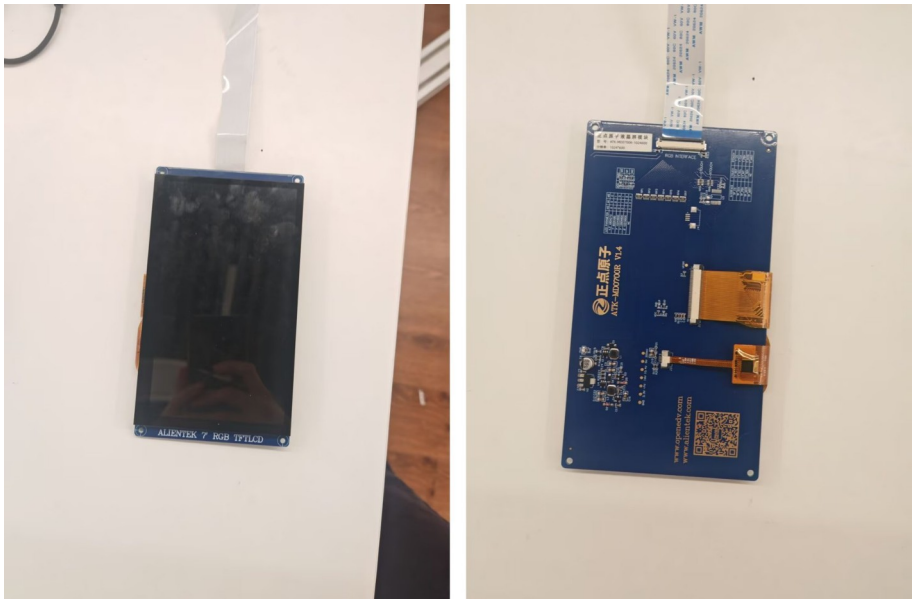


图 3-4 ATK-MD0700R 7 寸触摸屏正反面

主控器件为 XC7Z020CLG400-2。PS 运行 Linux、交互服务、图像处理、轨迹状态机与机械臂通信；PL 集成逆解、显示时序、触摸接口、网口桥、识别预处理和音频接口。显示分辨率为 1024×600。完整平台的资源、时序和板端自检结果汇总于 3.3.8 节。

3.1.3 上位机软件

本系统的用户交互软件部署在开发板上，以本地网页作为触摸屏界面，无需额外电脑承担规划。手机网页提供扫码上传等辅助入口。首页按书写、交互和文化学习组织 13

种模式（图 1-1）；后端统一完成内容读取、纸面排版、笔画处理、规划、预览与执行。实际功能按使用场景在 3.2 节展开。

3.2 功能演示

各项演示依次给出画面或作品、操作与效果说明，以及对应的测试结果。纸面成品用于展示实际绘制效果，软件截图用于展示交互流程；计算性能统一说明测试平台与统计口径。

3.2.1 毛笔书写

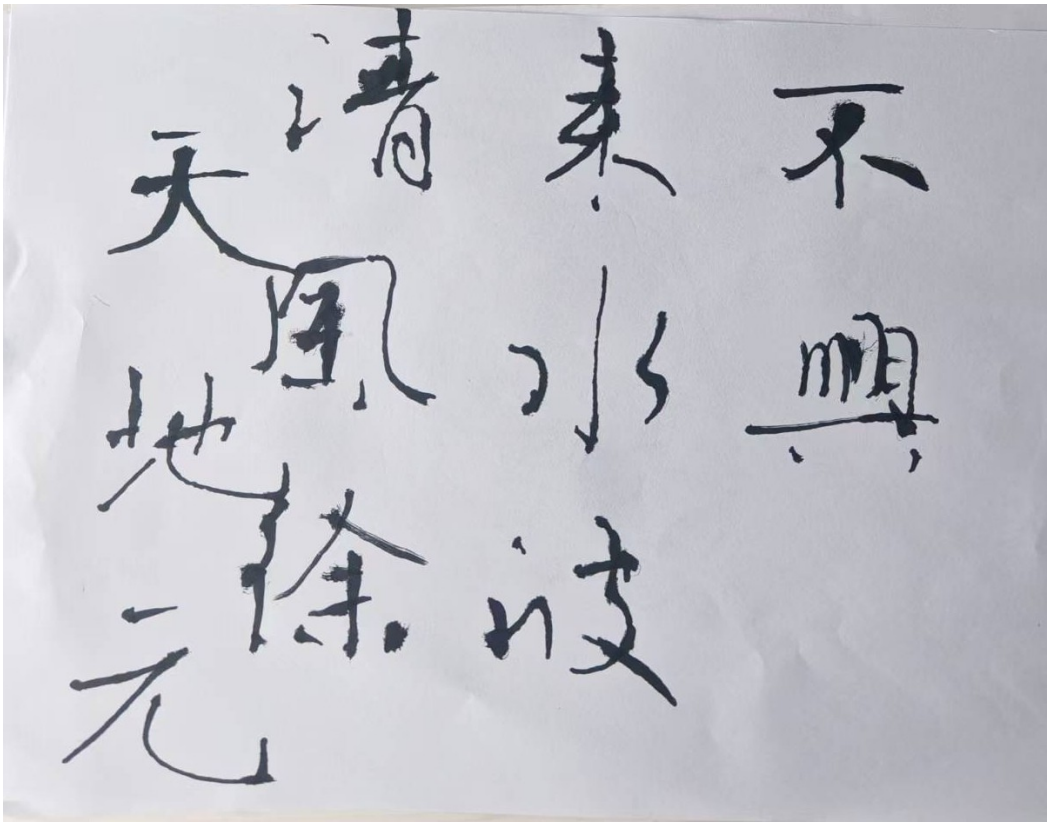


图 3-5　现场毛笔书写成品：多字排列与笔画形态

毛笔写字模式将汉字标准笔顺转为连续空间轨迹，按笔画依次完成抬笔、落笔和运笔。图 3-5 展示现场多字书写结果，可观察横、竖、撇、捺及转折的线条形态。系统支持 40–70 mm 字号，常用字库包含 3,500 字、33,959 笔；全部字形已完成离线规划检查，现场成果图展示其中实际书写的样例。

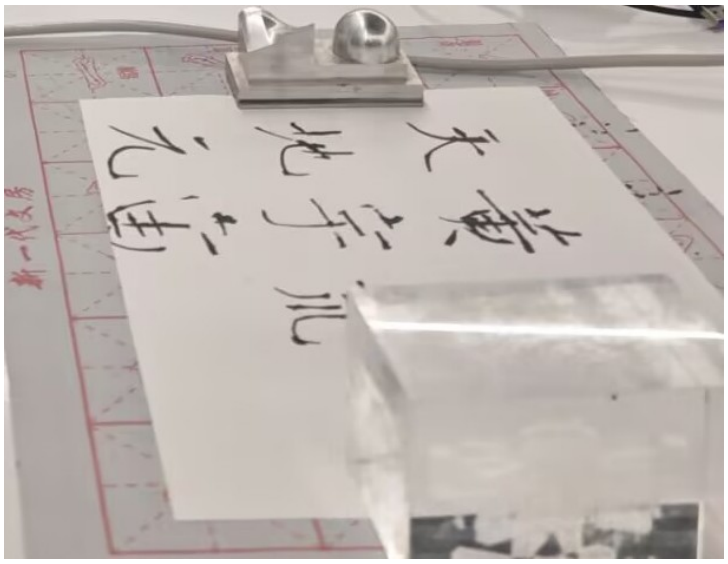


图 3-6　毛笔作品局部：天地玄黄、宇宙洪荒

3.2.2 硬笔书法与笔具切换



图 3-7　可更换的秀丽笔与中性笔

硬笔书法模式复用字形排版、连续规划和关节执行链路，通过更换笔具适配不同线宽。图中展示了实际使用的秀丽笔和中性笔。切换后需重新标定落笔高度，使笔尖接触条件与工具长度一致。本节展示实际使用的笔具配置。

3.2.3 人像线稿绘制

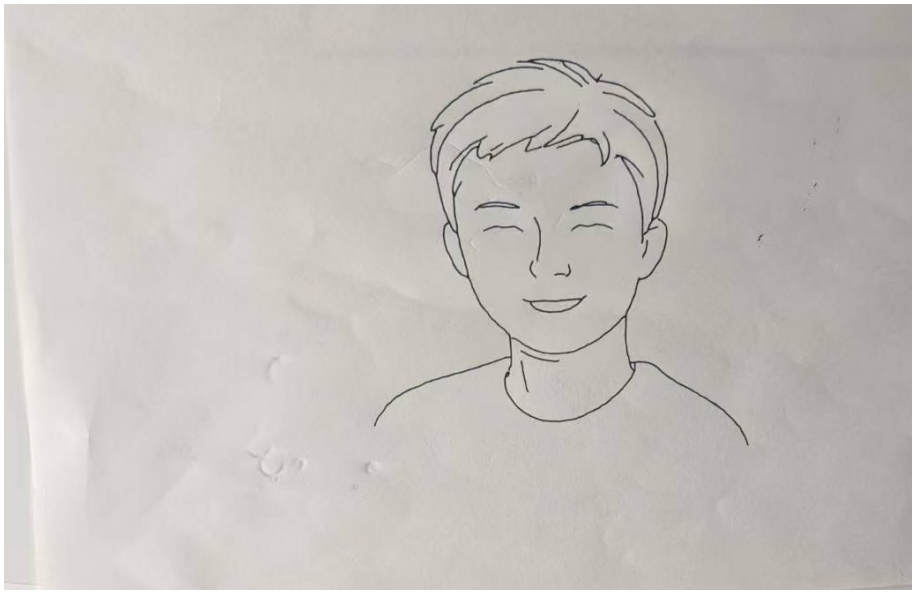


图 3-8　人像线稿纸面绘制成品

照片绘画流程支持手机扫码上传照片，经可选的云端图生图得到线稿，再在开发板上提取中心线、平滑路径并生成七轴轨迹；上传已有线稿可跳过云端。图 3-8 为本次提供的实际人像绘制成品，展示面部、发型、颈部和衣领轮廓，取代旧版本“正在规划”的界面图。该成品用于证明实物绘制效果；3.3 节的人像计时使用 T1、T2 数据集，不将其耗时直接套用于这张照片。

3.2.4 名帖表演、春联与诗词书写



图 3-9　当前首页的名帖、春联与诗词入口

名帖表演提供 7 部内容，并对共 21 列的规划结果进行板端预计算缓存；春联模式将上下联按列排版；诗词模式提供 40 首绝句。长文本按字号自动换行和分页，每页结束后归位并提示换纸。三种模式共享毛笔执行链路，区别在于内容组织与版式。当前图示为首页功能入口，多字毛笔实物效果见图 3-5。

3.2.5 笔画工坊与屏上手写



图 3-10　笔画工坊的部分模板预览（提按示意，非实测墨迹）

笔画工坊提供 88 种笔画模块，可调整字号及压深，用于分解演示运笔；屏上手写采集手指轨迹，经平滑、纸面映射与重定时后由机械臂复写。笔画模块在 20–60 mm 字号和压深组合下完成 616 项离线检查；屏上单笔的识别与规划在板端 12 组测试中，中位耗时由 884 ms 降至 751 ms。详细条件与曲线见 3.3.6、3.3.7 节。

3.2.6 手写识字与字迹识别

手写识字的板端测试结果见表 3-7。该模式将触屏输入转换为标准化字图，在板端离线识别 3,755 类汉字并显示候选结果。T7 的 300 张字体渲染样本中，部署模型的 Top-1 与 Top-5 均与原模型一致；采用双线程后单字识别中位耗时约 170 ms。字迹识别是独立的摄像头采集与比对评分入口，用于书写练习反馈；现有材料尚无可单列的真实字迹评分准确率，不以模型转换一致率替代。

3.2.7 诗文朗读、书法百科与趣味测验



图 3-11　诗文朗读：内容选择、逐句高亮与播放控制

诗文朗读支持内容选择、逐句高亮、暂停、停止、进度与音量控制；书法百科提供知识浏览，趣味测验提供题目交互。三种模式补充了书写演示之外的文化学习场景。朗读功能已有 62 项板端测试通过，扬声器实际出声尚未完成验收。百科与测验现有材料以软件功能实现为依据，尚无用户教学效果量化结果。

表 3-1　功能与演示证据对应关系

功能模式	对应演示	验证结果或范围
毛笔写字	图 3-5、图 3-6	纸面实物；3,500 字离线规划
硬笔书法	图 3-7	笔具实物；待补硬笔字成品
照片绘画	图 3-8	人像纸面实物；性能另见 T1/T2
名帖、春联、诗词	图 3-9	内容与界面；待补完整分项作品
笔画工坊、屏上手写	图 3-10	616 项离线检查；单笔板端计时
手写识字、字迹识别	3.2.6 节、表 3-7	识字转换一致性与板端计时；评分待量化
朗读、百科、测验	图 3-11 及图 1-1	软件交互；朗读 62 项测试，出声待验收

3.3 性能指标

3.3.1 实验平台与测试集

全部板端实验在 ATK-DF7020 上进行（PYNQ，Linux 5.4.0，Python 3.8.2），均未

驱动机械臂，只测计算；界面、摄像头等生产服务保持运行，因此计时含真实的系统负载。每组规划实验连续运行 3 次取平均，PL 平台的一次性加载时间单独记录。测试集见表 3-2。

表 3-2　测试集

编号	测试内容	样本规模
T1	照片绘画人像	20 笔；265 段；3,434 节点
T2	人像片段	8 笔；1,569 节点
T3	完整轨迹（4 阶段）	21,680 节点；21,679 个目标
T4	线稿图库	14 张；1024×1024
T5	手写单笔	4 种笔画 ×3 种尺寸
T6	笔画模块	88 种；共 616 项
T7	字体渲染样本	150 字 ×2 字体 =300 张

T1 用于阶段计时与 CPU/PL 对比，落笔长 517.3 mm；T2 用于早期对照；T3 用于精度与连续性；T4 用于提取加速；T5 为横、竖、撇、横折，尺寸 40/60/100 mm；T6 覆盖 5 种字号与压深组合，用于模块误差检查；T7 用于模型转换一致性。

评价指标：规划墙钟时间与各阶段时间；逆解吞吐；位置与姿态误差（独立 FP64 正解为参考）；相邻点最大关节步长；资源与时序；识别一致率与时延。凡是模型内计算的误差，都只说明“算得准不准”，不代表笔尖实际落在纸上的位置。

3.3.2 功能验证汇总

表 3-3　功能验证情况

功能	验证内容与结果	性质
七轴逆解核	RTL 协同仿真 192/192 点 + 13 个接口合约事务通过	仿真
逆解精度	T3 共 21,679 个目标 0 回退，位置误差最大 0.39 μm	离线，模型内
平台上板	开机加载新平台，规划核、识别、显示、触摸、PL 网口自检通过	板端实测
启停平稳	软停、加减速、保持自动模式、手写平滑：91 项测试通过	板端测试
手写识字	300/300 与原模型一致；现场书写识别正常	板端实测
诗文朗读	62 项测试通过；扬声器出声未验收	板端测试 / 待验收
软件回归	245 项通过、0 跳过（控制器为模拟）	离线

表 3-3 汇总了各功能的验证方式。其中“仿真”“离线”两类只说明计算正确，实物效果另行标注；尚未验收的项目在表中直接写明。

3.3.3 逆解精度与连续性

用 T3 检验目标 R2、R3：以初始节点为种子，21,679 个目标分 339 批连续送入 FP32 核（与板上核同一份 HLS C++ 源码在电脑上运行），每批以上一批最后一个关节解为种子，不用历史解重置分支。结果见图 3-12 与表 3-4。

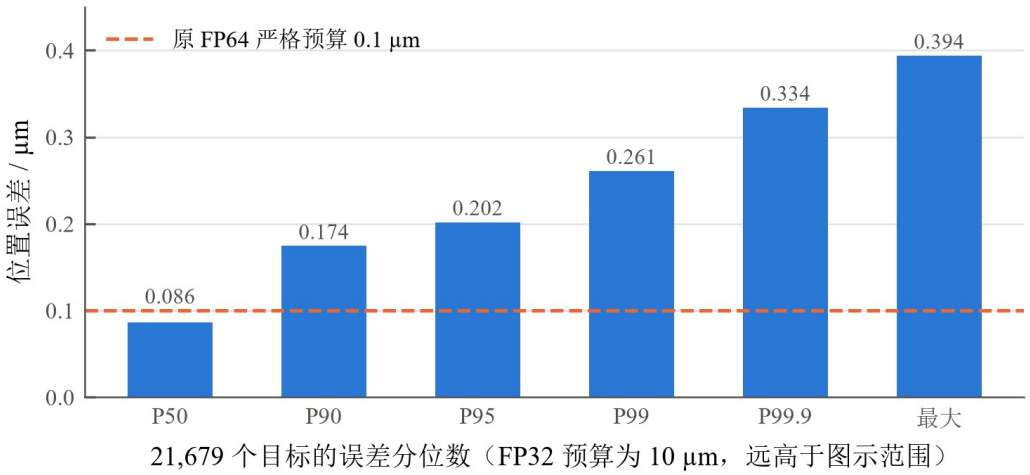


图 3-12　FP32 逆解位置误差分位数（T3，独立 FP64 正解为参考）

表 3-4　各运动阶段的逆解误差与关节步长

阶段	目标数	严格预算通过率	位置误差 μm	姿态误差 μrad	关节步长 mrad
1	5,448	59.9%	0.394	0.913	2.881
2	6,784	58.6%	0.370	1.010	1.555
3	2,582	58.9%	0.344	1.102	2.955
4	6,865	59.4%	0.367	1.048	1.617
合计	21,679	59.2%	0.394	1.102	2.955

后三列均为最大值。原严格预算通过数：阶段 1–4 分别为 3,265、3,975、1,522、4,076，合计 12,838。原预算为 0.1 μm / 1 μrad；FP32 验收预算为 0.01 mm / 100 μrad。

三点观察。第一，全部目标满足 FP32 预算（0.01 mm / 1×10^{-4} rad），最大位置误差 0.39 μm，比预算小两个数量级，R2 达到。第二，只有 59.2% 的目标满足原 FP64 实现的严格预算（0.1 μm / 1 μrad），不达标的几乎全部出在位置上（姿态单项 21,676/21,679 满足），这是单精度的固有代价，我们按 FP32 预算验收，两组数字分开报告。第三，最大关节步长 2.955 mrad，距关节限位的最小余量 0.720 rad，全程没有跳支，R3 达到；与历史冗余解相比单关节最大差 0.264 rad（约 15.1°），说明同一笔尖轨迹下肘部选了另一条同样连续的分支，而不是误差。

3.3.4 规划性能剖析

用 T1 在 10 月 24 日平台（v2c 数值迭代核）上分别以纯 CPU 和 CPU+PL 运行规划，各阶段耗时见图 3-13。CPU+PL 模式规划 3 次平均 10.84 s，纯 CPU 为 8.20 s，PL 模式反而慢 32%，差别全部来自“七轴连续逆解”一项（4.30 s 对 2.20 s），其余阶段两种模式基本相同。

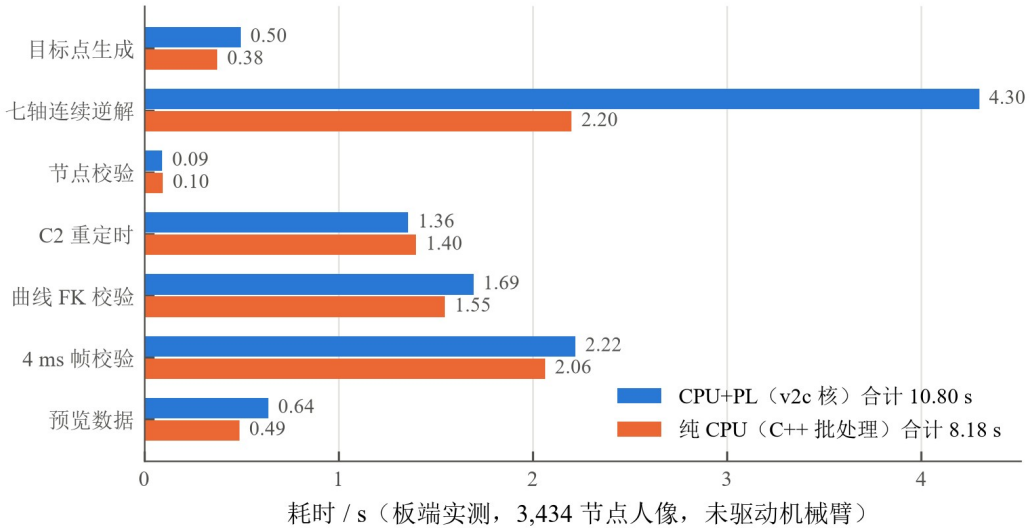


图 3-13　规划各阶段耗时（T1，板端实测）

把逆解环节进一步拆开（图 3-14），PL 批处理共 2.35 s，其中硬件实际计算的等待时间只有 0.29 s（3,433 个目标，折合每点约 85 μs），写入 DDR 与读回解析合计 1.41 s，占 60%；而 CPU 的 C++ 批处理完成同样工作只需 0.43 s。PL 路径的耗时可分解为式（3-1），瓶颈在主机侧的数据打包与解析，而不在硬件。

$$t_{IK}^{PL}=t_{wr}+t_{hw}+t_{rd}+t_{ctl}=0.50+0.29+0.91+0.65=2.35\,s \tag{3-1}$$

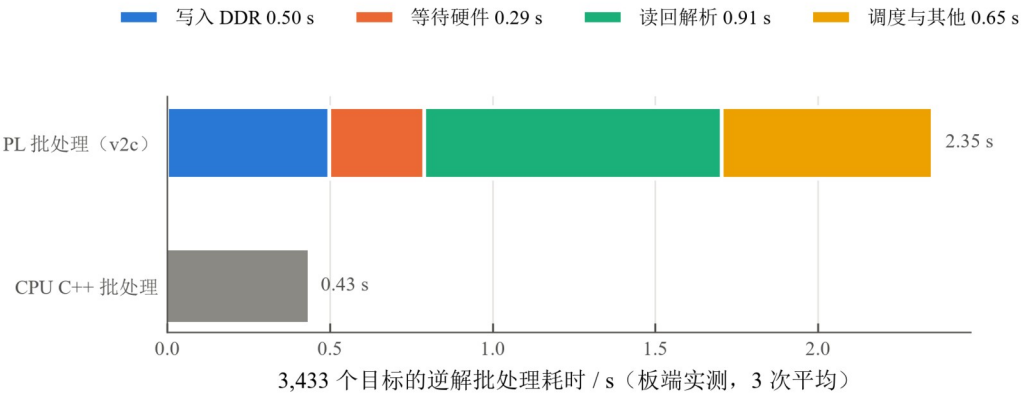


图 3-14　逆解批处理耗时构成（T1，板端均值（3 次））

整条“照片→可执行”计算链的热运行耗时见表 3-5。其中“线稿加速器加载尝试”6.20 s 是平台缺少细化 IP 时仍去尝试加载造成的浪费；FP32 版本已改为检测到无 IP 直接选 CPU，同条件下观测节省 6.68 s。FP32 版本还删除了曲线 FK 校验与 4 ms 帧校验两个环节，二者在本次实测中合计 3.91 s（PL 模式）。这些删减的实际收益要在 FP32 平台上重测后才能确认，本文不预先给出新的端到端数字。

表 3-5 “照片→可执行”计算链热运行耗时（T1，10 月 24 日平台，3 次平均）

环节	耗时 / s	说明
线稿提取	10.86	含线稿加速器加载尝试 6.20 s、CPU 细化 2.86 s
运动规划（CPU+PL）	10.84	逆解 4.30 s、曲线 FK 校验 1.69 s、4 ms 帧校验 2.22 s 等
规划结果冻结	0.22	
SDK 关节段准备	1.12	357 个执行块
预览复制与 SVG 生成	4.15	预览点 74,593 个
合计	27.18	另有一次性 PL 会话加载 12.0 s

3.3.5 线稿提取加速

T4 的 14 张图在旧线稿平台上测得图 3-15。整幅提取中位耗时由 5.88 s 降到 1.14 s（5.1 倍），其中细化子阶段中位由 2.19 s（OpenCV/NumPy）降到 0.32 s（C++）和 0.12 s（PL），图追踪由 1.25 s 降到 0.27 s（C++）。14 张图的输出路径与纯 CPU 结果逐项一致。当前平台为容纳识别核与 FP32 核已移除细化 IP，因此细化退回 CPU；把 C++ 细化接入这条回退路径是下一步工作。

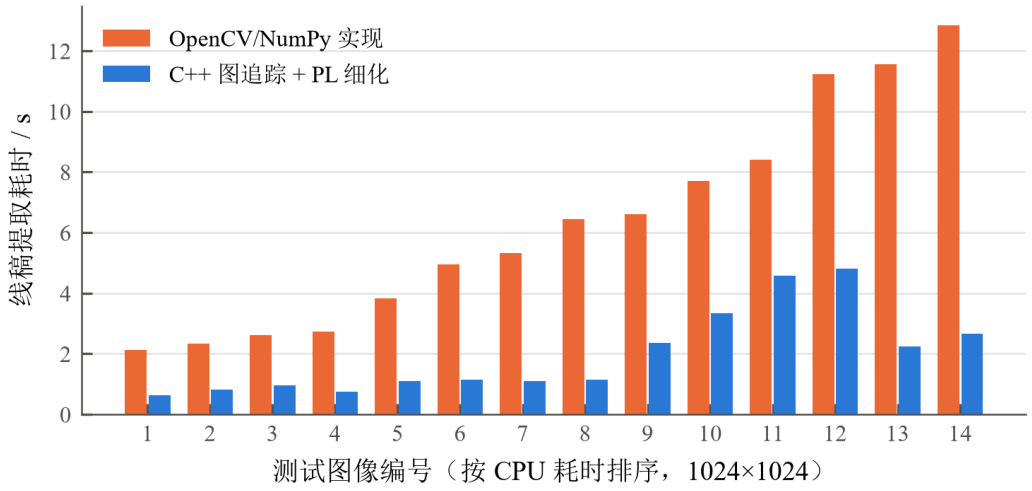


图 3-15 线稿提取耗时（T4，板端实测）

3.3.6 手写交互时延

T5 测的是从“写完一笔”到“关节段生成完毕”的计算时间（不含触摸采集、网络下发与机械臂起动），结果见图 3-16。12 组的中位为 751 ms（优化前 884 ms），识别只占 10.5–17.2 ms，其余为规划。耗时随笔画尺寸近似线性增长，带转折的横折最慢，100 mm 横折为 1,168 ms，超出 1 s 的交互目标（R4 部分达到）。模型内误差：落纸段

不超过 0.0199 mm，空中段不超过 0.154 mm。

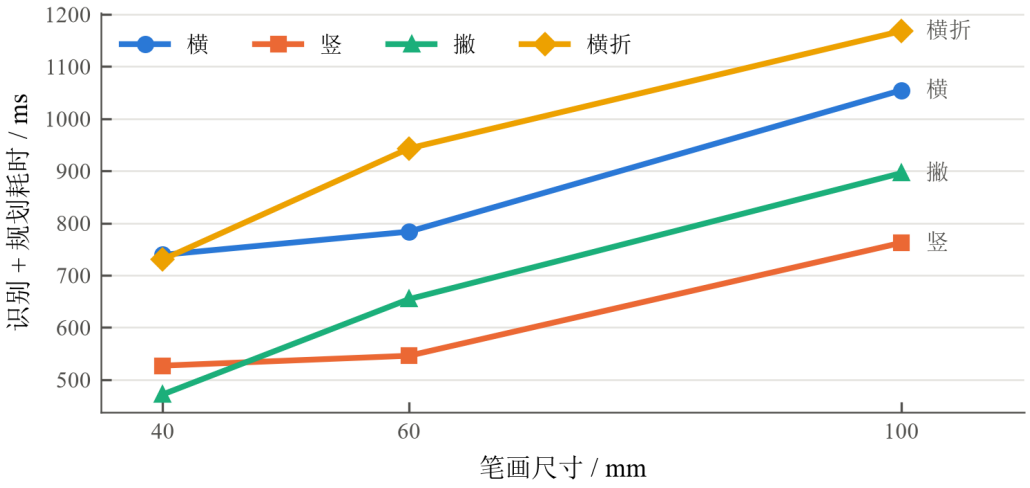


图 3-16　手写单笔识别 + 规划耗时（T5，板端实测）

3.3.7 笔画模块与手写识字

T6 共 616 项全部通过，按字号的统计见表 3-6。最大偏差 0.0238 mm 出现在 40 mm 字号的大压深组，这是规划结果与模板之间的偏差，不是实际落笔精度。

表 3-6　笔画模块规划检查（T6，离线）

字号 mm	项数	平均节点	平均耗时 s	最大偏差 mm
20	88	452	0.369	0.0091
30	88	523	0.385	0.0103
40（三档压深）	264	595	0.402	0.0238
50	88	678	0.421	0.0116
60	88	762	0.439	0.0112

手写识字的板端性能见表 3-7。T7 的 300 个样本上，板端 OpenCV 与原 PyTorch 模型的第 1 名和前 5 名全部一致，说明格式转换没有改变计算结果；它不代表手写识别准确率，真实触屏书写的准确率还需采集样本测量。

表 3-7　手写识字板端性能（300 个样本）

指标	单线程	双线程（采用）
模型加载时间	0.44 s	0.43 s
单字平均耗时	199.9 ms	177.0 ms
单字中位耗时	195.0 ms	170.1 ms
95% 分位耗时	222.9 ms	215.2 ms
常驻内存增加	49.5 MB	49.4 MB

3.3.8 平台资源与时序

表 3-8　完整平台实现结果（Vivado 2024.2，布线后）

资源 / 检查项	使用量 / 结果	占用率
LUT	40,048 / 53,200	75.28%
FF	56,343 / 106,400	52.95%
BRAM	67.5 / 140	48.21%
DSP	94 / 220	42.73%
Slice	13,261 / 13,300	99.71%
WNS / WHS	+0.136 ns / +0.011 ns	—
TNS / THS	0 / 0	—
DRC	0 错误	—

器件 xc7z020clg400-2。方法学告警保留在原始报告中。平台已开机加载，规划核 100 MHz，以及显示、触摸、PL 网口和识别自检通过。

3.3.9 加速方法迭代与性能对比

逆解加速经历了四个阶段，每一阶段都以板端或仿真实测为依据。

阶段 1：CPU 端算法与数据组织。最初规划核心逐点计算，8 笔人像需 157.16 s；改为 C++ 批处理后降到 13.33 s（11.8 倍）。自碰撞检查也改为 C++ 批处理并在两个 A9 核上并行，整幅人像规划由 23.6 s 降到 12.5 s。这一阶段与 FPGA 无关，但它建立了后续比较的基线，也让纯 CPU 成为一个很强的对手。

阶段 2：数值迭代核 v1 → v2c。第一版 PL 核用雅可比矩阵与 7×7 线性方程迭代求解。自写 sin/cos、改用 3×4 刚体变换、流水化雅可比，并用前两点线性外推作初值后，单次迭代由 22,214 周期降到 5,847 周期，每点约 0.92 ms 降到 0.10 ms（HLS 估计），4 万次随机对比关节差不超过 2.7×10^{-10} rad。

阶段 3：端到端实测给出反例。把核接入整条规划后，v1 使 8 笔人像由 10.72 s 变为 15.72 s；v2c 只快 2.4%（14.121 s → 13.787 s）；在 T1 上 v2c 比纯 CPU 慢 32%（3.3.4 节）。阶段计时和阿姆达尔模型（2.4 节）给出两个结论：逆解只占规划的 26.9%；PL 路径的时间主要花在数据搬运上。

阶段 4：FP32 解析核与去除重复复核。据此改变方向：（1）用 SRS 解析逆解替代数值迭代（2.5.3 节），计算改为 FP32，反正切改为定点 CORDIC，时钟提到 100 MHz；（2）删除运行时重复的曲线 FK 复核与密集帧 FK 探针，改为开发阶段用独立 FP64 正解验证（假设 H4）；（3）执行准备不再按 FK 误差合并节点，在主机上由 100.6 ms 降到 3.7 ms，但命令数由 78 条增至 557 条，对控制器队列的影响需上板测量。各版本对比见表 3-9，端到端与子阶段对比见表 3-10。

表 3-9 七轴逆解核各版本对比

版本	时钟 MHz	LUT	DSP	时延与口径
v1	76.92	约 26.5k	113	约 0.92 ms/点（HLS 估计）
v2c	76.92	30,298	67	约 85 μs/点（板端等待均摊）
FP32	100	24,159	48	约 8.6 μs/点（RTL 仿真）

算法：v1 为雅可比迭代与 7×7 求解；v2c 加入自写三角函数、3×4 变换、流水雅可比和外推初值；FP32 为 SRS 解析、CORDIC 选解与检查写出。v1/v2c 的单次迭代分别为 22,214/5,847 周期。FP32 的 64 点事务为 55,043–55,123 周期。v2c、FP32 资源为布线后结果；各版本时延口径不同，不直接计算统一加速比。

表 3-9 中三个时延口径不同：v1、v2c 的“每点”来自 HLS 周期估计，v2c 的 85 μs 是板上硬件等待时间除以点数（含批开销），FP32 的 8.6 μs 来自含仿真 AXI 读写的 RTL 协同仿真。它们只作量级参考，不折算为单一加速倍数，也都不是整幅规划时间。

表 3-10　板端端到端与子阶段对比（均未驱动机械臂）

优化项目	优化前	优化后
C++ 批处理（2 笔）	11.56 s	1.05 s（11.0×）
C++ 批处理（8 笔）	157.16 s	13.33 s（11.8×）
整幅人像并行规划	23.6 s	12.5 s
PL v1（8 笔人像）	10.72 s	15.72 s（变慢）
PL v2c（8 笔人像）	14.121 s	13.787 s（1.024×）
PL v2c（T1）	8.20 s	10.84 s（变慢）
关节段准备	96.86 s	5.86 s（缓存 1.15 s）
线稿提取（T4）	5.88 s	1.14 s（5.1×）
手写单笔（T5）	884 ms	751 ms

对照说明：①两组 C++ 批处理为 CPU 软件优化；②整幅人像含 5,267 节点，采用 C++ 自碰撞批处理与双核并行，输出逐字节不变；③v1 调度与复核开销超过核收益；v2c 的 8 笔测试中，硬件等待 0.122 s，分发/复核 2.614 s；T1 的数据搬运占逆解环节 60%；④关节段准备含 21,680 节点，去除重复计算并缓存，执行块逐一相同；⑤T4 为 C++ 图追踪与 PL 细化的历史结果，当前平台已移除细化核；⑥T5 为识别与规划优化。全部计时均未驱动机械臂。

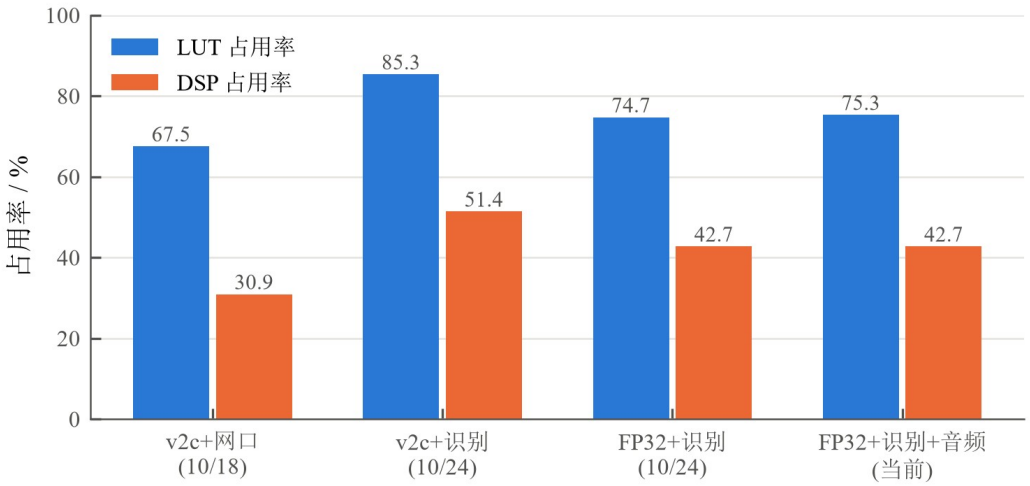


图 3-17　各版本完整平台 LUT 与 DSP 占用率（布线后）

FP32 解析核较 v2c 减少 6,139 个 LUT、19 个 DSP，时钟提高到 100 MHz，整板 LUT 占用由 85.32% 降至 75.28%（图 3-17）。全部目标满足 FP32 预算，59.2% 满足原 FP64 严格预算。FP32 的整幅规划时间、DDR 争用和 CPU 回退比例仍待板端测量，因此不报告其端到端加速倍数。保留 FK 复核时，逆解无限加速对应的规划时间模型下限为 5.98 s（图 2-7）。

第四部分　总结

4.1 可扩展之处

（1）在 FP32 平台上按 T1 重新计时，用实测数据确定 CPU/PL 分工阈值；（2）把主机侧打包与解析改为零拷贝的连续缓冲，压缩式（3-1）中的 t_{wr} 与 t_{rd}；（3）为关节段执行建立“几何节点→实际下发段→校验”专用路径，避免生成最终不用的 4 ms 全时序；（4）按几何误差自适应采样；（5）手写识字结果直接交给机械臂书写，形成“写—认—再写”闭环；（6）对空中轨迹做 S 型速度插补，进一步减小起停冲击。

4.2 心得体会

第一，测量比直觉可靠。我们最初以为“逆解放进 FPGA 就会快”，第一版核接入后整幅规划反而从 10.72 s 变成 15.72 s。分阶段计时显示，硬件真正计算只占很小一部分，时间都花在数据搬运、逐点复核和后续校验上。用阿姆达尔模型算一下就明白：逆解只占四分之一多一点，单靠它最多快 1.37 倍。这次反例让我们把精力从“继续堆核”转向“减少不必要的数据通路和重复计算”，后来的 FP32 解析核也因此做得更小。

第二，资源接近满载时，每个模块都要算账。完整平台的 Slice 占用达到 99.71%，加入识别核时只能拿掉线稿细化核；换成解析逆解后才腾出空间加入音频。我们学会了用布线后的层次化报告逐模块核对，而不是只看 HLS 的估计数字；HLS 报告的时钟估计也不能当作时序收敛的证据。

第三，“DRC 零错误”不等于外设正常。音频首版的 DRC 没有错误，但 SDA 的输入和三态缓冲在综合时丢失，总线常低并产生虚假应答，让我们一度把编解码器误认成 WM8960。最后是检查布线后网表、并对 I²C 地址做真实探测，才确认实际芯片是 ES8388。这次教训被写进了封装前的网表检查。

第四，与大模型协作要有边界和证据。智能体写代码、跑测试、整理记录的效率很高，但也会自信地给出错误判断。我们用交接提示词写清目标、必读资料和“已验证事实”，用证据台账记录每个数字的出处和性质，并规定智能体不代输密码、不擅自让机械臂运动。凡是写进报告的数字，都要能追溯到原始日志或报告（附录 C 的表 C-3）。

第五，安全始终优先。机械臂会在规划完成后自动运动，开始前必须有人守在急停旁；停止改为减速软停，故障时仍立即停止；任务之间保持自动模式以避免下沉。这些都是在实机调试中一次次碰到问题后补上的。

4.3 大模型工具使用声明

本作品在开发过程中使用了代码智能体（OpenAI Codex、Anthropic Claude Code），主要用于代码编写与调试、测试与构建脚本、过程记录与文档整理。所有实机操作、方案取舍与最终验收由团队成员完成；报告中的数字均来自工程中的原始日志与报告，详细使用情况见附录 A。

第五部分　参考文献

[1] Buss S R. Introduction to inverse kinematics with Jacobian transpose, pseudoinverse and damped least squares methods[R]. 2004.

[2] Nakamura Y, Hanafusa H. Inverse kinematic solutions with singularity robustness for robot manipulator control[J]. Journal of Dynamic Systems, Measurement, and Control, 1986, 108(3): 163-171.

[3] Shimizu M, Kakuya H, Yoon W K, et al. Analytical inverse kinematic computation for 7-DOF redundant manipulators with joint limits and its application to redundancy resolution[J]. IEEE Transactions on Robotics, 2008, 24(5): 1131-1142.

[4] PYNQ Project. PYNQ: Python Productivity for Zynq[EB/OL]. https://pynq.readthedocs.io.

[5] AMD. Zynq-7000 SoC Technical Reference Manual (UG585)[Z]. 2023.

[6] Everest Semiconductor. ES8388 Low Power Stereo Audio CODEC Datasheet[Z].

[7] Goodix. GT911 Capacitive Touch Controller Programming Guide[Z].

[8] Sandler M, Howard A, Zhu M, et al. MobileNetV2: Inverted residuals and linear bottlenecks[C]// Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2018: 4510-4520.

[9] Liu C L, Yin F, Wang D H, et al. CASIA online and offline Chinese handwriting databases[C]// International Conference on Document Analysis and Recognition. 2011: 37-41.

[10] ONNX. Open Neural Network Exchange[EB/OL]. https://onnx.ai.

[11] Bradski G. The OpenCV Library[J]. Dr. Dobb's Journal of Software Tools, 2000.

[12] Zhang T Y, Suen C Y. A fast parallel algorithm for thinning digital patterns[J]. Communications of the ACM, 1984, 27(3): 236-239.

[13] Amdahl G M. Validity of the single processor approach to achieving large scale computing capabilities[C]// Proceedings of the AFIPS Spring Joint Computer Conference. 1967: 483-485.

[14] Siciliano B, Sciavicco L, Villani L, et al. Robotics: Modelling, Planning and Control[M]. London: Springer, 2009.

[15] Volder J E. The CORDIC trigonometric computing technique[J]. IRE Transactions on Electronic Computers, 1959, EC-8(3): 330-334.

[16] Andraka R. A survey of CORDIC algorithms for FPGA based computers[C]// Proceedings of the ACM/SIGDA Sixth International Symposium on Field Programmable Gate Arrays. 1998: 191-200.

[17] Biagiotti L, Melchiorri C. Trajectory Planning for Automation Machines and Robots[M]. Berlin: Springer, 2008.

[18] AMD. Vitis High-Level Synthesis User Guide (UG1399)[Z]. 2024.

[19] 全国大学生嵌入式芯片与系统设计竞赛组委会. 2026 FPGA 赛道选题指南（AMD）[Z]. 2026.

[20] AMD. Vivado Design Suite User Guide: Design Analysis and Closure Techniques (UG906)[Z]. 2024.

附录 A　大模型协作记录

A.1 协作方式

开发中使用代码智能体（OpenAI Codex、Anthropic Claude Code）。人负责确定目标、取舍方案、所有实机操作（上电、急停、输入 sudo 密码）和最终验收；智能体负责阅读工程、编写代码与脚本、运行测试、HLS 与 Vivado 构建，并把过程写成记录。三条固定规则：（1）每次交接都写交接提示词，列出目标、必读资料、“已验证的事实（不要重新怀疑）”、禁区和剩余工作；（2）所有对外数字记入证据台账，注明数值、性质、来源文件和“不能怎么说”；（3）智能体不代输、不保存任何密码，不擅自发送运动指令。典型记录见表 A-1。

A.2 提示词与回答记录（节选）

表 A-1　典型提示词、回答与验证

阶段	协作记录
规划提速	**用户原文 / 提示要点：**“编译烧录成功，但规划超过两分钟”，附现场失败日志 **智能体的回答与采纳：**区分规划慢、起始逆解停滞和执行停止三类问题；PS 几何计算批量化，增加分阶段计时 **验证：**同输入 3 次主机基准中位 6.795 → 1.320 s，关节角零差异；声明不是板端实测
编译故障	**用户原文 / 提示要点：**“队友反映 FPGA 部分编译失败，请检查” **智能体的回答与采纳：**先索取首条 ERROR、工具版本与核名，本地复现 Tcl 缺文件；改绝对路径、显式 C++11、隔离测试台 **验证：**106 项回归通过；明确“未运行 AMD HLS”
加速路线	**用户原文 / 提示要点：**“规划下一步 FPGA 加速，尽量让规划时间缩短到商业水平” **智能体的回答与采纳：**把“商业水平”转为可测目标（固定输入热规划 P95 ≤ 1 s）；测得批槽利用率仅 14.57%，提出跨线段合批 **验证：**形成 64 点批处理接口；不宣称已达到目标
七轴交接	**用户原文 / 提示要点：**交接提示词：逆解必须在 ZYNQ 本地并用 FPGA 加速；用关节角驱动机械臂 **智能体的回答与采纳：**完成 v2c 核、平台封装与板端测速流程 **验证：**v2c 平台 WNS +0.209 ns；8 笔人像 CPU/PL 对比实测

阶段	协作记录
耗时审计	**用户原文 / 提示要点：**只读审计：板端分工、FPGA 资源与规划耗时 **智能体的回答与采纳：**指出 PL 只覆盖部分计算，重复的全时序校验才是主要开销；给出 P0–P6 建议 **验证：**形成 FP32 解析核与去除重复复核的方案
音频调试	**用户原文 / 提示要点：**接手诗文朗读部署；音频帧计数为 0 **智能体的回答与采纳：**从布线网表发现 SDA 缓冲丢失，改显式 IOBUF；用地址探测确认 ES8388 **验证：**XSim TB_PASS，时序收敛；62 项板端测试通过，声音待验收

A.3 自我纠错轨迹

表 A-2 列出开发中被证据推翻的判断，以及推翻后的修正。

表 A-2 自我纠错记录

初始判断	证据与修正
“逆解进 PL 端到端一定更快”	**发现问题的证据：**8 笔人像 CPU 10.72 s，CPU+PL 15.72 s；T1 上 8.20 s 对 10.84 s **修正：**阶段计时 + 阿姆达尔模型，转向解析核与去除重复复核
编解码器是 WM8960	**发现问题的证据：**0x1A 的应答来自 SDA 常低造成的伪应答；修复后探测到 0x10 **修正：**改写 ES8388 初始化；不再用 0x1A 作为身份证据
DRC 零错误即可封装	**发现问题的证据：**首版网表 SDA 缺 IBUF/OBUFT，告警为 Warning 级别 **修正：**增加布线网表缓冲检查，缺失即拒绝封装
FP32 结果与原算法“逐位一致”	**发现问题的证据：**严格预算下仅 59.2% 通过 **修正：**改按 FP32 预算验收，两组数字分开记录
手写空中段也做平滑	**发现问题的证据：**空中段平滑后偏离规划路径 23.75 mm **修正：**只平滑书写段，空中段精确到位
按 FK 误差合并执行节点	**发现问题的证据：**合并结果未经 FK 验证 **修正：**保留全部细节点（557 条），准备耗时 100.6 → 3.7 ms
编译日志编码错误后补签结果	**发现问题的证据：**中文编译日志编码导致编排退出 **修正：**保留失败运行，修复后重新建立运行，不补签旧结果

A.4 智能体工作流

每一轮工作按以下步骤进行，产物保存在工程 docs/ 与 build/ 目录：（1）读交接提示词与必读资料，先核对现场状态，不重复怀疑已验证事实；（2）只读剖析，写出问题清单与建议顺序，由人确认后再改动；（3）改动前备份，每次核修改依次做 A/B 对比、C 仿真、综合、RTL 协同仿真、Vivado；（4）上板按暂存 → 哈希校验 → 激活 → 自检执行，失败即回滚，需要 sudo 时请人执行；（5）把结果写入交付记录与证据台账，标明哪些已实测、哪些仅仿真、哪些尚未验收，并更新下一轮交接提示词。交接提示词的骨架如下：

```
## 0. 目标          —— 用户的明确要求，逐条编号
## 1. 必读资料      —— README、模块说明、上轮记录
## 2. 硬件与网络    —— 地址表；不要替用户输入或保存任何密码
## 3. 已验证的事实（不要重新怀疑）—— 唯一可动路径、落笔高度、已知坑
## 4. 已完成的代码 —— 文件 + 内容 + 测试命令
## 5. 当前状态与可继续的优化
## 6. 剩下的工作（按顺序）—— 每步的验收标准与回滚方法
```

附录 B　技能包提炼过程

B.1 提炼来源

技能包（仓库 skill/ 目录）只收录在本项目中真实踩过、并有记录可查的问题，来源见表 B-1。

表 B-1　技能包条目来源

经验来源	提炼条目
多轮智能体交接中反复重查已验证事实、误用旧参数	**提炼出的条目**：交接提示词模板（含“已验证事实”“禁区”） **类型**：提示词工作流
对外材料中出现夸大或口径混淆的数字	**提炼出的条目**：证据台账模板与“禁止说法”清单 **类型**：提示词工作流
加速核单测更快、接入后整体更慢	**提炼出的条目**：阶段计时脚本 + 阿姆达尔估算表：先算上限再决定是否硬件化 **类型**：校验脚本
PL 核接口反复调整、失败处理不一致	**提炼出的条目**：批处理 HLS 核接口模板：64 点批、取消字、成功前缀 + 首个失败点 **类型**：案例模板
上板后屏幕或网口异常需要快速恢复	**提炼出的条目**：PYNQ 硬件包发布脚本：暂存 → 校验 → 激活 → 自检 → 回滚 **类型**：校验脚本
音频 SDA 缓冲丢失但 DRC 无错误	**提炼出的条目**：布线网表 I/O 缓冲检查；时序/DRC 验收戳 **类型**：校验脚本
板载浏览器、线程事件循环、Windows 长路径等问题	**提炼出的条目**：踩坑清单 **类型**：踩坑清单

B.2 目录结构

```
skill/
├── README.md                适用范围、使用方法、失效条件、已验证效果
├── prompts/
│   ├── handoff_template.md     交接提示词模板
│   └── evidence_ledger.md      证据台账模板与禁止说法
├── templates/hls_batch_kernel/ 批处理核接口模板（HLS + PYNQ 驱动）
├── checks/
│   ├── stage_profile.py        分阶段计时与阿姆达尔上限估算
│   ├── overlay_release/        stage / verify / activate / rollback
```

```
│   ├── netlist_io_buffers.tcl 布线网表 I/O 缓冲检查
│   └── build_verified.py      WNS/WHS ≥ 0、DRC 0 错误才生成验收戳
└── pitfalls.md                踩坑清单
```

B.3 验证方法与效果

（1）发布流程：历次板端发布（joint、3500 字库、vision、styles、FP32 音频平台）均按“备份—哈希校验—安装—自检—回滚预案”执行，每次保留备份与哈希清单，出现异常时可用一条命令回退；（2）网表检查：修复版音频网表经检查确认 SDA 为 IBUF + OBUFT 后才封装，首版网表作为反例用于自测；（3）阶段计时：T1 的计时结果直接给出了 26.9% 的逆解占比与 1.37 倍上限，改变了优化方向；（4）证据台账：本报告中的所有数字均能在台账中找到出处与性质（表 C-3）。

B.4 适用范围与失效条件

适用：基于 PYNQ 的 Zynq-7000 项目；用 HLS 做批处理加速核并需要 CPU 回退的场景；多人或多智能体接力开发。失效条件：①非 PYNQ 的裸机或 PetaLinux 工程，发布脚本需改写；②流式实时处理（非批处理）不适用批接口模板；③网表检查只覆盖 I/O 缓冲，不能代替时序约束检查与外设实测；④证据台账依赖人如实填写来源，不能自动判断数字真伪。

附录 C　复现说明

C.1 环境与工具版本说明

复现所需环境见表 C-1。

表 C-1　复现环境

项目	版本 / 型号
开发板	正点原子 ATK-DF7020（XC7Z020CLG400-2）
FPGA 工具	Vivado / Vitis HLS 2024.2（Build 5239630）
板端系统	PYNQ 镜像，Linux 5.4.0-xilinx-v2020.2，Python 3.8.2
电脑端	Windows 10/11，Python 虚拟环境 .venv
外设	ATK-MD0700R 显示屏、USB 摄像头、天链 TCB705 机械臂（可选，无机械臂时用模拟模式）

关于工具版本：选题指南[19]推荐 Vivado/Vitis 2026.1 或 2025.2，本作品使用 2024.2。项目启动时开发环境已为 2024.2，全部 HLS 核都在该版本下完成 C 仿真、RTL 协同仿真和 IP 打包；完整平台 Slice 占用 99.71%，更换版本需要重新做时序收敛[20]，赛前风险较大；板端 PYNQ 只加载比特流与 HWH 文件，与 Vivado 版本无关。为保证可复现，仓库提供全部构建脚本（scripts/build_tcb705_parallel_fp32_hls.ps1、scripts/build_tcb705_parallel_fp32_vivado.ps1、hw/parallel/create_fp32.tcl、build_fp32.tcl），以及已生成的 bit / hwh / xsa 与 SHA-256 清单。

C.2 获取代码

开源地址：【提交前填写仓库地址】，许可证：MIT。仓库目录均为英文名；密钥配置文件（config/secrets.local.json）不随仓库发布，使用者按 config/*.example.json 自行填写。

C.3 从零复现步骤

步骤 1：电脑端演示与测试（无需硬件）。建立 Python 虚拟环境后运行 run_tcb705_demo.bat，浏览器打开 http://127.0.0.1:8095/ 进入模拟模式，不发送任何运动。运行单元测试：

```
.\.venv\Scripts\python.exe -m unittest tests.test_tcb705 tests.test_tcb705_web
tests.test_tcb705_lineart_plan
```

步骤 2：HLS 构建逆解核（含 C 仿真、综合、RTL 协同仿真与 IP 导出）：

```
./scripts/build_tcb705_parallel_fp32_hls.ps1 -BuildDirectory C:\fpga_build\p32_rebuild `
   -Profile controller_throughput -Stage all
./scripts/build_tcb705_parallel_fp32_hls.ps1 -BuildDirectory C:\fpga_build\p32_rebuild `
   -Profile contract -Stage all
```

步骤 3：Vivado 构建完整平台（使用短路径目录，避免 Windows 子 IP 路径过长）：

```
./scripts/build_tcb705_parallel_fp32_vivado.ps1 -HlsDirectory C:\fpga_build\p32_rebuild `
   -ProjectDirectory C:\fpga_build\p32_full_rebuild -Stage all
```

核对 export 目录下 timing_summary.rpt（WNS、WHS ≥ 0）、drc.rpt（0 错误）和 build_verified.json，再用网表检查确认音频 SDA 为 IBUF + OBUFT。

步骤 4：封装与上板。将 bit、hwh 与寄存器元数据封装为 overlay 目录（platform.json 含哈希），用 deploy/tcb705/ 下的 stage_fpga_release.sh → verify_fpga_release.sh → activate_fpga_release.sh 上板，失败时运行 rollback_fpga_release.sh。需要 sudo 的步骤由操作者本人输入密码。

步骤 5：部署应用与服务。按 deploy/README.md 部署应用代码并安装 systemd 服务；在板上运行：

```
python -m unittest discover -s tests -p 'test_tcb705*.py'
```

步骤 6：实机书写（可选）。连接机械臂控制器网线，确认急停可及、周围无人；在屏幕上选择“毛笔写字”，输入文字后开始。机械臂会先归位，再书写，结束后再次归位。

C.4 预期输出与核对点

各步骤的预期结果见表 C-2。

表 C-2 复现核对点

步骤	预期结果
电脑端测试	测试全部通过（需板上 DLL 的少数用例在电脑端跳过）
HLS	C 仿真、RTL 协同仿真通过；64 点事务约 5.5 万周期
Vivado	WNS、WHS ≥ 0，DRC 0 错误，生成 bit / hwh / xsa
上板自检	开机日志出现规划核协议自检、识别自检通过；规划核 100 MHz、未报故障
规划结果	规划指标中 ik_backend 显示 PL 批处理后端，并记录 PL 批次数与 CPU 回退数

C.5 数据与证据索引

报告中的关键数字与原始文件的对应关系见表 C-3，路径相对于工程根目录。

表 C-3　关键数字证据索引

编号	数字与来源
E01	**数字：**DH 参数、关节限位；21,679 点误差分位与阶段统计 **来源：**hw/reports/parallel_fp32_20261024/full_trajectory/summary.json
E02	**数字：**协同仿真 55,043–55,123 周期；192/192 点；atan 1,041,041 组 **来源：**hw/reports/parallel_fp32_20261024/README.md、validation.json
E03	**数字：**整板资源与时序（40,048 LUT，+0.136/+0.011 ns） **来源：**hw/reports/audio_iobuf_20261027/utilization*.rpt、build_verified.json
E04	**数字：**v2c 平台 45,392 LUT；逆解模块 30,298 LUT / 67 DSP **来源：**docs/板端分工_FPGA 资源与规划耗时审计_20261024.txt
E05	**数字：**T1 阶段计时、CPU 8.20 s / PL 10.84 s、传输分解 **来源：**build/runtime_profile_20261024/measurements.json、summary.json
E06	**数字：**8 笔人像 14.121 / 13.787 s **来源：**build/tcb705/deployment/v2c_transport_profile.json
E07	**数字：**8 笔人像 10.72 / 15.72 s；11.0× / 11.8× **来源：**build/tcb705/benchmark_arm_pl.json、benchmark_arm_cpu.json
E08	**数字：**关节段准备 96.86 → 5.86 s **来源：**build/tcb705/startup_speed_20261020/board_benchmark.json
E09	**数字：**线稿提取 14 张图 **来源：**build/tcb705/deployment/lineart_acceptance/gallery_pl_patch1.json
E10	**数字：**手写单笔 751 / 884 ms **来源：**build/handwriting_20261023/board-benchmark-optimized.json、board-benchmark.json
E11	**数字：**笔画模块 616 项 **来源：**build/calligraphy/preset_validation_v4.json